

Volatility Modelling and Regime Classification Methods

An understanding exercise

Eloy Sentana Seguí

September 2025

1 Introduction

Market calm never lasts forever. Periods of low uncertainty often give way to sudden turbulence, due to the very human nature of markets reacting to worldwide events. This rhythm of stability and shock is what makes volatility so central to finance. It defines how investors price risk, how institutions manage portfolios, and how we interpret what's happening in the world economy.

As we will see throughout this practice, volatility tends to cluster, meaning that quiet markets and stormy markets each have a kind of memory. Models like GARCH capture this persistence by linking today's volatility to the magnitude of recent shocks as well as recent volatility.

Still, real markets go through regime changes that no single model can fully explain. The calm before 2008 was not the same as the calm of 2019. Hidden Markov Models and Markov Switching frameworks aim to uncover these hidden shifts—moments when the underlying behavior of the market changes.

1.1 Motivation

The purpose of this notebook is simple: to learn by doing. I wanted to get hands-on with modern methods for volatility estimation and forecasting, especially GARCH models and regime-switching approaches such as Hidden Markov Models (HMM) and Markov Switching Autoregressive (MSAR) models. Rather than just reading about them, I wanted to implement these models, observe how they behave with real financial data, and understand what their results actually mean.

Along the way, I have tried not only to reproduce results but also to question them. Why this distribution and not another? Why this algorithm, this test, this assumption? Exploring these “whys” often led me beyond the main task and into the theorems behind the models, the statistical tests that support their assumptions, and the broader principles of mathematical modeling in finance. Those detours have proved just as valuable as the direct results.

This notebook doesn't follow strict academic conventions: it doesn't include full citations or detailed proofs. My goal was to explore, experiment, and build understanding of how volatility can be modeled and forecasted. Working with real data and algorithms has highlighted something important: results in practice rarely match the clean outcomes in research papers. That gap between theory and reality has been one of the most interesting parts of the journey, pushing me to ask better and deeper questions.

I began from the work of [Yunhui Wang](#), but extended it significantly. I have added my own implementations, experiments, and reflections, turning this notebook into a kind of learning journal, where curiosity and questioning are just as central as coding and results.

1.2 Disclaimer

This notebook has not been supervised by any academic or professional of any field. It is purely a personal learning exercise and should not be taken as financial advice or a definitive guide to volatility modeling. If you spot any errors, have any questions, suggestions, or corrections, please feel free to reach out to esentanasegui@gmail.com

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Disclaimer	2
2	Data	4
2.1	Simple Return vs Log	4
2.2	Distribution of prices and returns	6
2.2.1	Hypothesis testing for mean return and distribution normality	8
3	Volatility Modelling with GARCH	9
3.1	Definition and Types of Volatility	9
3.1.1	Realized Volatility	9
3.1.2	Implied Volatility and the VIX Index	9
3.2	Timeframes and Volatility	10
3.3	Introduction to the GARCH model	11
3.4	Long-term variance under GARCH(1,1)	15
3.5	Conditional volatility	16
4	Future volatility forecasting with GARCH	20
4.1	Blind volatility prediction	20
4.2	Continuously Trained GARCH Model	23
4.3	Train-and-Run Approach	26
4.4	Comparison of Volatility Estimation Methods	29
4.5	Comparing Conditional Volatility with VIX	29
5	Regime Classifiers	31
5.1	Introduction to Markov Chains	31
5.2	Hidden Markov Model	32
5.3	Markov Switching Autoregression Model	34
5.4	Comparison	36
6	Conclusion	37
7	References	38

2 Data

Throughout this practice, we will use the daily adjusted closing prices of the S&P 500 index (^GSPC), VIX Index from September 1st 2005 to October 1st 2025. The data is sourced from Yahoo Finance and can be easily accessed using the `yfinance` library in Python.



Figure 1: SP500 Closing Price History

Although raw prices provide context, they are not suitable for most statistical models because prices are non-stationary (their statistical properties change over time), and their magnitude changes a lot (especially in long timeframes).

Therefore, we should use the daily returns, from which we have two options: Simple vs Log returns.

2.1 Simple Return vs Log

The simple return for a price series is defined as:

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

where:

- R_t : simple return at time t
- P_t : price at time t
- P_{t-1} : price at time $t - 1$

The logarithmic return (log return) for a price series is defined as:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

where r_t is the log return at time t .

Simple returns are intuitive, but log returns have properties that make them preferable for modeling and compounding. Here is a good example to understand log returns:

Let's suppose a stock has the following (simple) returns for three days:

$$r_1 = 5\%, \quad r_2 = -3\%, \quad r_3 = 4\%.$$

With simple returns, if we want to know the total return after the three days, we should do:

$$P_3 = P_0(1 + r_1)(1 + r_2)(1 + r_3).$$

If $P_0 = 100$, then:

$$P_1 = 100 \times 1.05 = 105,$$

$$P_2 = 105 \times 0.97 = 101.85,$$

$$P_3 = 101.85 \times 1.04 = 105.924.$$

The total simple return is:

$$\frac{P_3 - P_0}{P_0} = \frac{105.92 - 100}{100} = 5.924\%.$$

Note that we cannot simply add $5\% - 3\% + 4\% = 6\%$ because the compounding doesn't allow for addition.

However, with log returns:

$$R_1 = \ln(1.05) = 0.048, \quad R_2 = \ln(0.97) = -0.0304, \quad R_3 = \ln(1.04) = 0.0392.$$

Now, the cumulative log return is just the sum:

$$R_{\text{total}} = R_1 + R_2 + R_3 = 0.05755166...$$

Very important: It is the LOG cumulative return. The corresponding price is:

$$P_3 = P_0 e^{R_{\text{total}}} = 100 \times e^{0.05755166} = 105.924$$

Which is exactly the compounded simple return, but here the cumulative return was obtained by simply *adding* the log returns.

Another interesting property of taking the log returns is that we are downplaying the big positive returns, and exaggerating the big negative returns. The effects can be seen with a scatter plot (Figure 2).

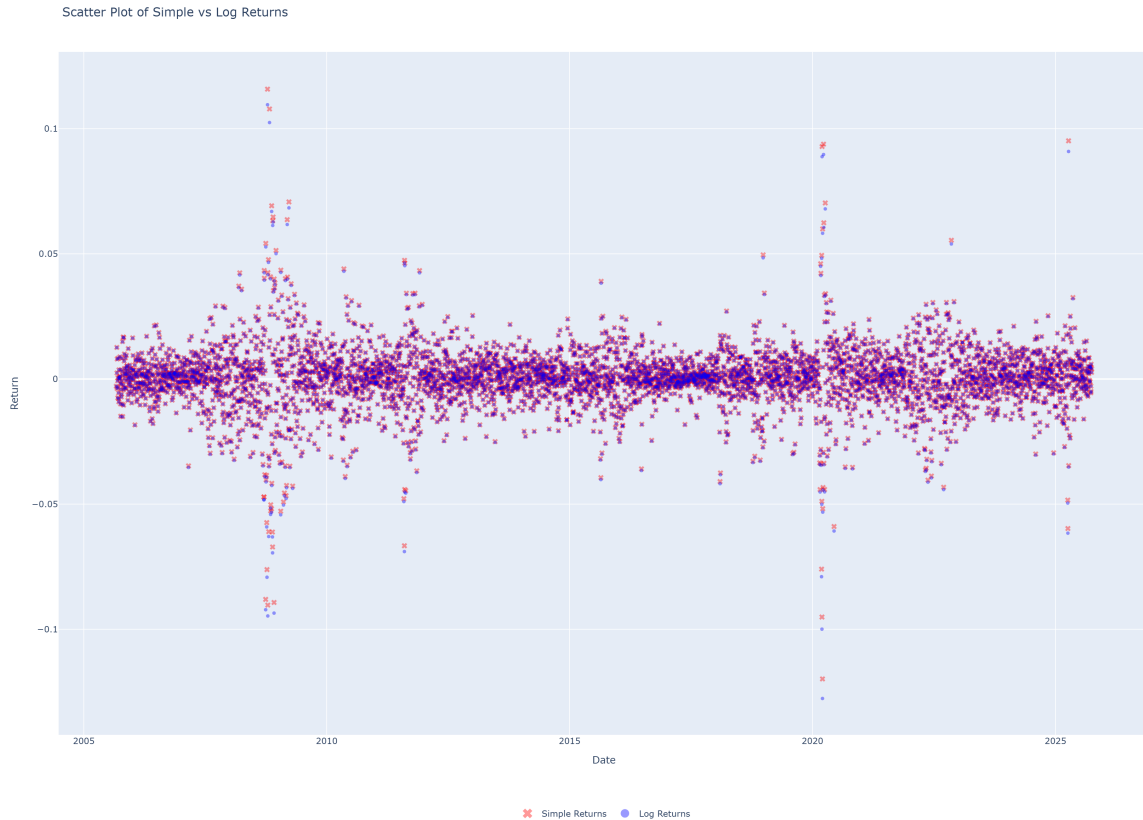


Figure 2: Simple vs Log Returns Scatter Plot

2.2 Distribution of prices and returns

Throughout the financial literature, prices are assumed to follow a log-normal distribution. However, in practice, this is not always the case (especially on shorter timeframes). See Figure 3.

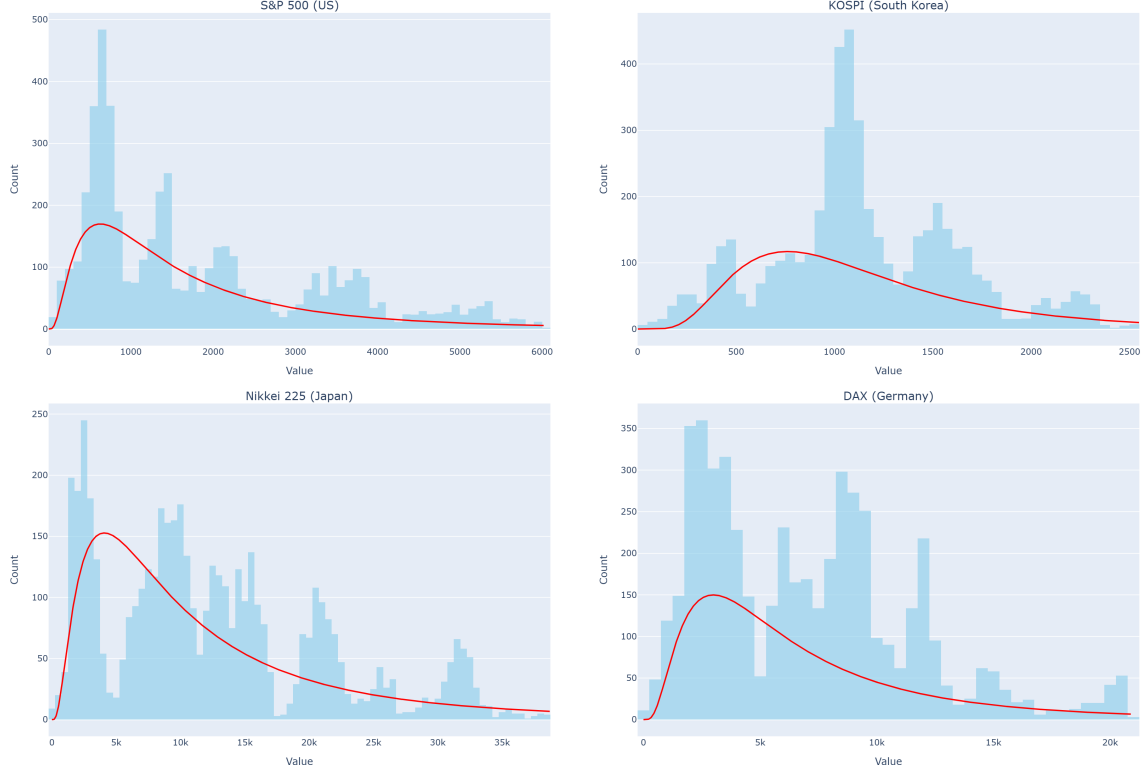


Figure 3: Best-fit Log-Normal Distribution of Prices

If we assume continuously compounded (log) returns $r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$ are normally distributed, then prices P_t must follow a log-normal distribution because:

$$P_t = P_0 e^{r_1 + r_2 + \dots + r_t}$$

and the exponential of a normal variable is log-normal.

In reality, returns have fatter tails, negative skewness and have 3+ kurtosis (leptokurtic).

Several reasons support the log-normal assumption for prices:

- Prices cannot fall below 0 and (in theory) can go up to $+\infty$, so the support of the log-normal distribution matches that of the prices.
- If prices are log-normal, then log returns are normally distributed, which is mathematically convenient for other financial models that assume normality (like the Black-Scholes).
- For small daily changes (around $\pm 3\%$), simple and log returns are nearly identical.
- Over long periods, log returns are easier to aggregate since they are additive across time.

Nevertheless, deviations from the normality assumption become pronounced when returns are extreme:

1. For very high returns, the log return is lower than the simple return.
2. For very negative returns, the log return is even lower, due to the asymmetric transformation.

Now that we know what distribution prices (are assumed to) follow, we can dive into that of returns. Do returns follow a normal distribution? Figure 4 and Table 1 clearly tell they don't.

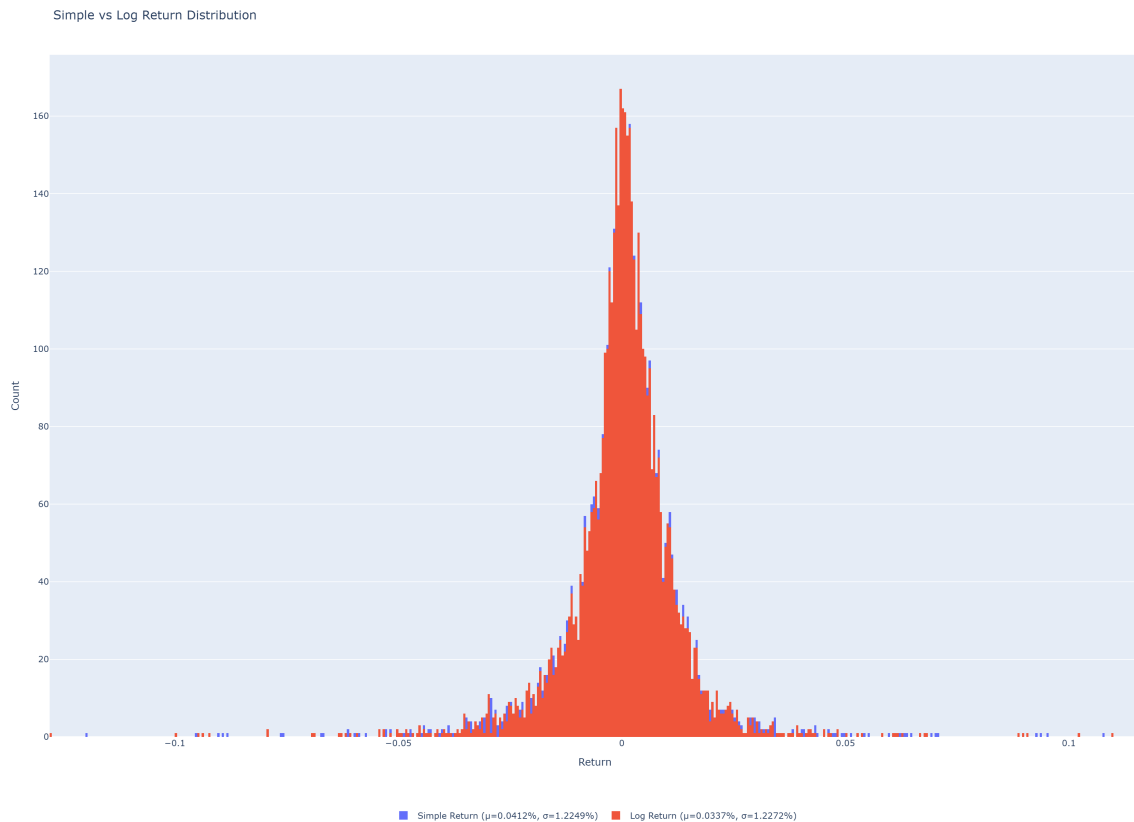


Figure 4: Distribution of Simple vs Log Returns

Table 1: Summary Statistics of Simple and Log Returns

	Simple Return	Log Return
Mean (%)	0.0412	0.0337
Std Dev (%)	1.2249	1.2272
Skewness	-0.2047	-0.4752
Kurtosis	12.7805	12.9504

From Table 1 above, we can interpret the following:

- The mean daily return is slightly positive, i.e. on average the returns tend to be slightly positive.
- The standard deviation is relatively high, i.e. significant day-to-day volatility (1.22% daily volatility, on average).
- The negative skewness means that large negative returns are more frequent than large positive returns.
- The high kurtosis (for a normal distribution it's 3) indicates fat tails in the return distribution, meaning extreme returns (both positive and negative) occur more often than would be expected under a normal distribution.

But... aren't log returns supposed to make the distribution more normal? Why is the skewness even more negative when taking the log returns? This is because the simple returns already have negative

skewness, and since the log returns compress the upside and stretch the downside, the skewness becomes even more negative.

So in this case, the simple returns distribution is (slightly) "closer" to being a normal distribution (less kurtosis, skewness closer to 0). This is actually surprising!

2.2.1 Hypothesis testing for mean return and distribution normality

To check for normality, we will use the arch Python package for hypothesis testing.

First we will test whether the mean differs significantly from 0. After running `t_stat, p = stats.ttest_1samp(asset_return, popmean=0, alternative='two-sided')`, we got `p = 0.0503628`. Since our null hypothesis for the mean was that the mean of the return series is 0, if we use an alpha of 0.05, we cannot reject the null hypothesis (i.e. we have no significant statistical proof that the mean differs from 0). This result is very important, since throughout this practice (and in most of financial literature) we will assume that the mean of the returns is 0.

Let's now test for normality: `k2, p = stats.normaltest(return)`.

And we got `p = 3.68834e-258`, so we reject the null hypothesis that the distribution of the log returns is normal.

And this is why we will use GARCH in this practice! Because with GARCH we can use non normal, fat tailed returns!

3 Volatility Modelling with GARCH

To understand how GARCH models volatility, we first need to define what *volatility* actually means.

3.1 Definition and Types of Volatility

Volatility is a [measure](#) of the dispersion of an asset's returns over time. In finance, we usually distinguish between two main types:

- Realized (Actual) Volatility: computed from past price data, usually as the standard deviation or variance of returns over a given period.
- Implied Volatility: derived from option prices (with the [Black–Scholes](#) model for example) and reflects market expectations of future volatility rather than past fluctuations.

3.1.1 Realized Volatility

Realized volatility reflects how much the asset's price has fluctuated in the past. Figure 5 compares different methods used to compute it.

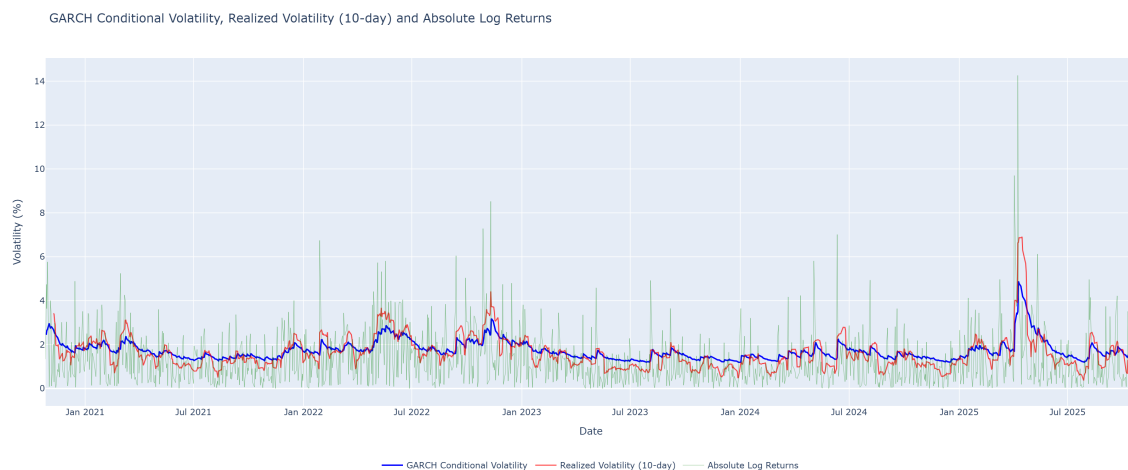


Figure 5: Comparison of Realized Volatility Estimation Methods

It is important to note that realized volatility cannot be directly observed for a single day, since volatility is defined as a dispersion measure over a period. (typically the standard deviation of a single data point is [undefined](#)). However, some proxies can be used to approximate daily realized volatility:

1. Using past returns over a short window (e.g., 5–15 days), which smooths volatility estimates.
2. Using the day's realized return (assuming mean 0), which yields a noisier measure.
3. Using GARCH or similar volatility models to obtain the *conditional variance* for each day.

3.1.2 Implied Volatility and the VIX Index

On the other hand, Implied Volatility is forward-looking, and it reflects the market expectation of future volatility. It doesn't reflect past dispersion of returns. The most famous measure is the VIX index, which estimates the market's expectation of 30-day annualized volatility for the S&P 500.

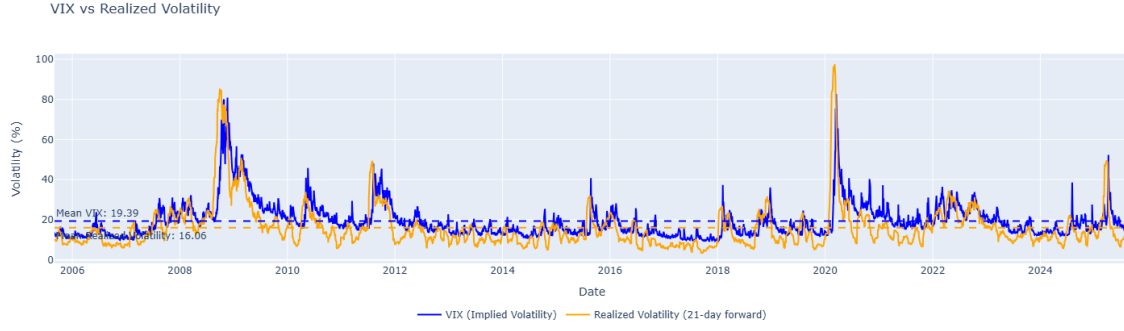


Figure 6: VIX vs. Realized Volatility (Annualized)

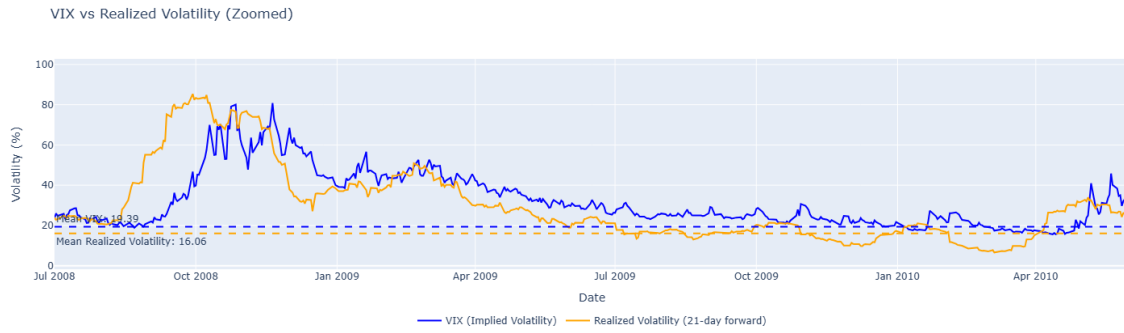


Figure 7: VIX vs. Realized Volatility during the 2008 Crisis

Figures 6 and 7 show the comparison between the 30-day (approx 21 trading days) forward realized volatility of the S&P 500 and the VIX. We see that the VIX generally overestimates realized volatility (by about 3.33%) especially on calm periods (except on big price movements, where the VIX is unable to anticipate them).

Since the VIX index is obtained from option prices, this spread between VIX and GARCH could be explained by investors paying a [premium](#) to [hedge](#) that risk.

3.2 Timeframes and Volatility

Volatility can be measured for a single day (that is, the volatility realized or expected for that specific day) or for a longer period, such as a week, a month, or a year.

Given a daily volatility value, we can obtain the volatility over a longer period by multiplying it by the square root of the number of days in that period:

$$\sigma_T = \sigma_d \times \sqrt{T}$$

where:

- σ_T is the volatility over the period of T days,
- σ_d is the daily volatility, and
- T is the number of trading days in the period (e.g., $T = 21$ for a month, $T = 252$ for a year).

Daily volatility usually ranges between 0.5% and 2%, while annual volatility typically ranges from 8% to 30%.

Unless stated otherwise, throughout this work I will use daily volatility in most plots, tables, and experiments.

3.3 Introduction to the GARCH model

The **GARCH** (Generalized Autoregressive Conditional Heteroskedasticity) model is used to estimate and forecast the volatility of time series, such as financial returns. It is an “advanced” version of the ARCH model introduced by **Robert Engle** (1982)

The basic idea is that the variance (σ^2 , squared volatility) at time t depends on past squared shocks (ε) as well as past variances. This means that volatility tends to cluster around periods of high volatility.

But what does GARCH mean?

1. Generalized: Extends the simpler ARCH model by including past variances as explanatory variables, not just past shocks.
2. Autoregressive: The current volatility depends on its own past values (as in any **AR models**).
3. Conditional: The variance is estimated with the information given at time $t-1$ (i.e., it is not constant, but conditional on history).
4. Heteroskedasticity: The variance of the error term changes over time (we will return to this later).

The general GARCH(p, q) formula is:

$$\sigma_t^2 = \omega + \sum_{i=1}^q \alpha_i \varepsilon_{t-i}^2 + \sum_{j=1}^p \beta_j \sigma_{t-j}^2$$

Where

ω	constant (baseline volatility)
σ_t^2	conditional variance at time t
α_i, β_j	parameters
ε_{t-i}^2	past squared shocks (errors)
σ_{t-j}^2	past variances

ω ensures that the variance never collapses to zero

But... what is the shock ε_{t-i}^2 ? Mathematically, returns are modeled as:

$$r_t = \mu_t + \varepsilon_t$$

That is, for every return, we try to predict it, but there is always some error (sometimes small, sometimes large) in our guess. That error is the shock.

And how do we get the expected return μ_t ? The answer is: it depends. Sometimes a constant is used; other times it is modeled with an **ARMA** process. In the case of a constant, it is obtained by “training” the model on past data, or getting the mean of all returns over a period. In some cases it is even set to 0 (in this case, we saw earlier that the mean was not significantly different from 0), because the main focus is volatility.

The shock is modeled as:

$$\varepsilon_t = \sigma_t z_t$$

where

σ_t	conditional volatility at time t (changes over time, modeled by GARCH)
z_t	a random variable with mean 0 and variance 1 (often assumed i.i.d. normal)

And why do we square the shock in the GARCH formula? Because volatility must always be non-negative, and we care about its magnitude, not its direction.

With this information, we can better understand the intuition behind the GARCH model: if yesterday's volatility was high, today's volatility is also likely to be high.

Luckily for us, the **best and most widespread** version of GARCH is the GARCH(1,1) model:

$$\sigma_t^2 = \omega + \alpha_1 \varepsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2$$

Here, today's volatility depends on yesterday's squared shock and yesterday's volatility.

Another important property of GARCH is “Covariance stationarity”. For this property, we need:

$$\sum_{i=1}^q \alpha_i + \sum_{j=1}^p \beta_j < 1$$

which ensures that the shocks eventually “die out,” and volatility returns to its long-run mean. If

$$\sum_{i=1}^q \alpha_i + \sum_{j=1}^p \beta_j = 1$$

it is called IGARCH (integrated), where volatility persists at any horizon. If

$$\sum_{i=1}^q \alpha_i + \sum_{j=1}^p \beta_j > 1$$

volatility explodes (i.e., it increases without bound, which is not the case for financial markets).

And what about the “Heteroskedasticity”? It simply means that the variance of the shock (error) is not constant across time. Let’s visualize this by first plotting the shock values over time (Figure 6), and then seeing their distributions: We will split the data in 4 periods, and see how the distribution of shocks changes accross time.

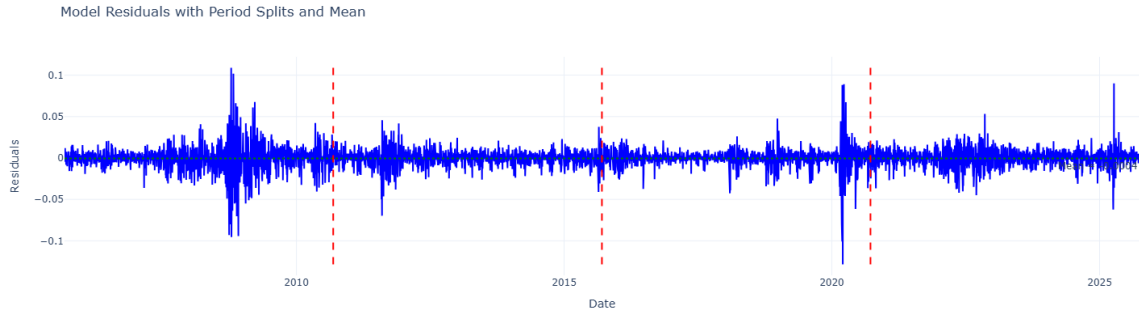


Figure 8: Residuals across time

On Figure 8 we can see the residuals (i.e. the shocks) plotted over time. If we remember, a return is defined as $r_t = \mu_t + \varepsilon_t$ where μ_t is the expected return (which is typically a constant), and ε_t is the shock (error) at time t. So this graph plots the “surprise” we got each day I.e. actual return - expected return. In this case, we used mean (expected return) = 0, so ($\varepsilon_t = r_t$)

We can see (Table2, Figure 9) that they are quite different across time! (variance of the first term is 2.5x the variance of the second term). This is mainly because some periods are calmer than others (i.e. less big surprises). The KDE plots also show how the distribution of shocks changes across time, with some periods having fatter tails (the KDE is just a smoothed line that takes the values of the histogram, does “mini” distributions and adds them up to get a smooth line).

Table 2: Variance across different periods

Period	Start	End	Variance
Period 1	2005-09-26	2010-09-23	0.000246
Period 2	2010-09-24	2015-09-22	0.000093
Period 3	2015-09-23	2020-09-18	0.000147
Period 4	2020-09-21	2025-09-22	0.000117

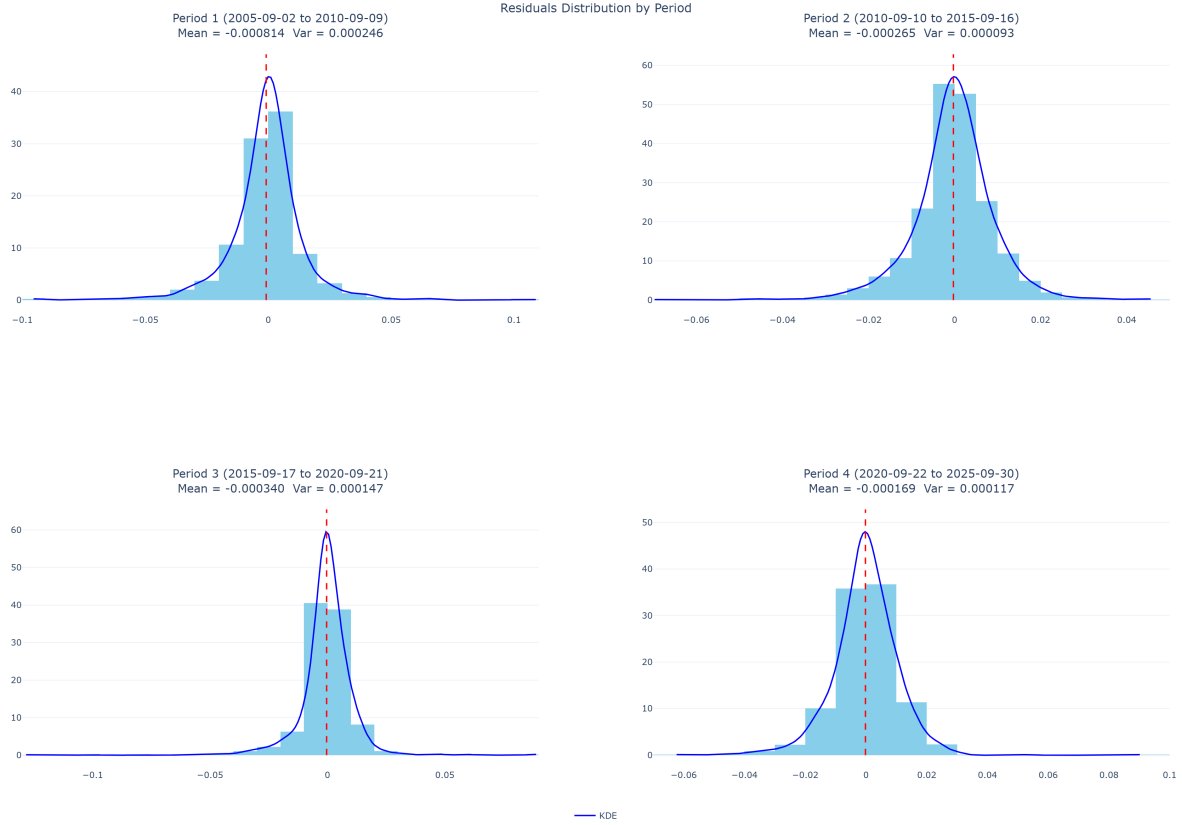


Figure 9: Distribution of shocks over the 4 periods

Let's analytically check that the variance is not constant, with Robert Engle's [Lagrange Multiplier method](#), a test developed by the creator of the ARCH model himself. It basically tells us that if we have a regressive model $y_t = X_t * \beta + \varepsilon_t$ (where X_t is the vector of regressors of the trained garch model we did earlier, and beta the parameter vector to optimize), then we formulate the following hypothesis:

- Null hypothesis (H0): No ARCH effects (constant variance).
- Alternative hypothesis : We have heteroskedasticity (i.e. changing variance).

We first estimate the model with [Ordinary Least Squares](#) (i.e. basically we try to find a line/hyper-plane that is closest to the data points, i.e. fitting) so that we have a relationship between the dependent variable y and the independent variable(s) x1, x2, etc. So it is essentially a minimization problem, where we try to minimize the sum of squared shocks (errors).

After getting this regression, we square the residuals (shocks), and then we build the following regression:

$$\hat{\varepsilon}_t^2 = \alpha_0 + \alpha_1 \hat{\varepsilon}_{t-1}^2 + \alpha_2 \hat{\varepsilon}_{t-2}^2 + \dots + \alpha_p \hat{\varepsilon}_{t-p}^2 + u_t$$

The $\hat{\varepsilon}_t^2$ are the squared residuals (shocks) we got from the first regression, and we regress them on their own lags (i.e. past values of themselves). The number of lags p is a parameter we choose!

Then we estimate again with OLS the alpha parameters, and if these params are jointly significant, i.e. if they are statistically significantly different from 0, then we reject the null hypothesis of homoskedasticity (constant variance), and we can conclude that there is heteroskedasticity (changing variance).

Once we have that regression, we get $R^2 = 1 - \text{SumSquaredResiduals} / \text{SumSquaredTotal}$ and then we compute the LM statistic:

$$LM = n \cdot R^2$$

, where n is the length of the return series. Therefore, if there are no ARCH effects, the $\hat{\varepsilon}_t^2$ should not be explained by its own lags, and R^2 should be close to 0. If there are ARCH effects, then the lagged

squared errors should explain a significant portion of the variance in the current squared errors, so the R^2 value must be higher.

$LM = n \cdot R^2$ follows a Chi-squared distribution (proved by Engle) with q degrees of freedom (where q is the number of lags we used in the regression). So we can get the p-value from this distribution, and if it is lower than 0.05 (i.e. 5% significance level), then we reject the null hypothesis of homoskedasticity (constant variance), and we can conclude that there is heteroskedasticity (changing variance).

The result of doing

```
test_stat, p_value, _, _ = statsmodels.stats.diagnostic.het_arch(res.resid)
```

is

```
ARCH test statistic: 1425.0719386593055
p-value: 3.828403737409418e-300
```

The ARCH test statistic is the $LM = n \cdot R^2$ value. The p-value is very small (below our significance level) so we reject the null hypothesis, and we can determine that there is heteroskedasticity (changing variance)!

With these plots (and the ARCH variance test), we can see that the variance is not constant accross the 4 periods (but this extends to any number of periods!). We see that the variances differ significantly, and it makes sense logically: On the first period, the 2008 crisis created more volatility (i.e. higher variance), on the second period, large shocks were rare (only the 2010 European debt crisis stands out, but in general it was a “calm” period). On the thrid period, the COVID-19 crisis increased volatility, and on the 4th period only the tariffs caused some shocks.

With this (both analytical and visual) analysis, we have explained why a model that takes into account the Heteroskedasticity of variances is important.

So, going back to the GARCH model, what about the α and β parameters? They are not set manually. During the “training”, they are estimated using maximum likelihood estimation (MLE). The model assumes returns follow a conditional variance process:

$$\sigma_t^2 = \omega + \alpha_1 \varepsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2$$

- ω : baseline variance
- α_1 : impact of recent shocks
- β_1 : persistence of past variance

MLE finds the parameter values that maximize the likelihood of the observed data, given the model. On this practice we use the arch Python package that does the parameter finding automatically. Thus, α and β are learned from the data to best fit the observed returns. Let’s thus build a model with the log returns and the arch Python library.

```
am = arch.univariate.arch_model(log_rets, x=None,
                                mean='constant', lags=None,
                                vol='Garch', p=1, o=0, q=1,
                                dist='normal', hold_back=None, rescale=True)

volatility_model = am.fit()
volatility_model.summary()
```

and we get:

```
=====
                        Constant Mean - GARCH(1,1) Model Results
=====
Dep. Variable: Close                                R-squared: 0.000
Mean Model:    Constant Mean                        Adj. R-squared: 0.000
Vol Model:     GARCH                                Log-Likelihood: -6785.30
```

```

Distribution: Normal                                AIC: 13578.6
Method:      Maximum Likelihood                    BIC: 13604.7
No. Observations: 5050
Date: Tue, Sep 23 2025                            Df Residuals: 5049
Time: 23:15:47                                    Df Model: 1
=====
Mean Model
-----
      coef      std err      t      P>|t|      95.0% Conf. Int.
mu      0.0734      0.0110      6.682      2.363e-11      [0.0519, 0.0949]
-----
Volatility Model
-----
      coef      std err      t      P>|t|      95.0% Conf. Int.
omega     0.0279      0.0055      5.089      3.604e-07      [0.0172, 0.0387]
alpha1    0.1358      0.0142      9.600      8.019e-22      [0.1080, 0.1640]
beta1     0.8430      0.0143     59.131      0.000e+00      [0.8150, 0.8710]
=====

```

3.4 Long-term variance under GARCH(1,1)

Let's now explore a concept that I find very interesting: What happens when we want to make a very far away prediction? Do we get the mean of all the variances filtered by our model? Or is there any other alternative?

Here are the parameters of our fitted GARCH(1,1) model:

Table 3: Estimated parameters of the GARCH(1,1) model

Parameter	Estimate
μ	0.073383
ω	0.027933
α_1	0.135800
β_1	0.843019

Let's break them down:

1. **mu**: This is the mean (expected return) of the time series
2. **omega**: This is the constant term in the variance equation (what makes volatility never go to 0)
3. **alpha[1]**: It measures how much yesterday's squared shock affects today's volatility
4. **beta[1]**: It measures how much yesterday's volatility affects today's vol. (higher beta = more persistence = more vol clustering)

Therefore our GARCH(1,1) model is defined by:

$$\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2$$

And with our values:

$$\sigma_t^2 = 0.027933 + 0.135800 \varepsilon_{t-1}^2 + 0.843019 \sigma_{t-1}^2$$

Long-term variance is the level of variance that the process will converge to if you let time go to infinity (do not mistake it with the omega value!). Indeed, the long term variance is defined with:

$$\sigma_\infty^2 = \frac{\omega}{1 - \alpha - \beta},$$

and since Volatility is the sqrt of the Variance:

$$\sigma_\infty = +\sqrt{\frac{\omega}{1 - \alpha - \beta}}.$$

Therefore, if we want a rough estimate of what will volatility do in a long time, we can take σ_∞ as an estimate! In fact, let's compute it with $\sigma_\infty = \omega / (1 - \alpha - \beta)$:

Long-term daily (scaled) variance with GARCH: 1.318741 / (annualized): 332.322849
 Long-term daily volatility with GARCH: 1.15 % / (annualized: 18.23 %)
 Sample daily volatility estimate: 1.23 % / (annualized: 19.48 %)

The long term volatility tells us that our model expects returns to fluctuate about $\pm 18.23\%$ per year around the mean ($\mu = 0.073383$).

Since the sample volatility is long-term GARCH volatility, it means that the model expects the future to be less turbulent than our test period. This tells us that GARCH is more forward looking than our sample estimate. This result makes sense, since our dataset included fairly volatile periods, and our alpha and beta values indicate high persistence ($\alpha + \beta$) of volatility.

3.5 Conditional volatility

We saw earlier that the Conditional Volatility is the volatility level we could expect at time t given all the information available up to the previous period $t - 1$.

After fitting the model (and obtaining the parameters), it goes back to the very first residual, and:

1. It initializes the first variance (usually with the sample variance or the unconditional variance):

$$\sigma_1^2 = \frac{\omega}{1 - \alpha - \beta}$$

2. Then, recursively, for each t , it uses the previous residual ε_{t-1} (observed) and the previous variance σ_{t-1}^2 (computed) to calculate the new σ_t^2 , according to the formula we saw earlier:

$$\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2$$

where $\omega > 0$, $\alpha \geq 0$, and $\beta \geq 0$ are the parameters estimated during the fitting process.

Figure 10 and Figure 11 show the absolute returns and the conditional volatility.

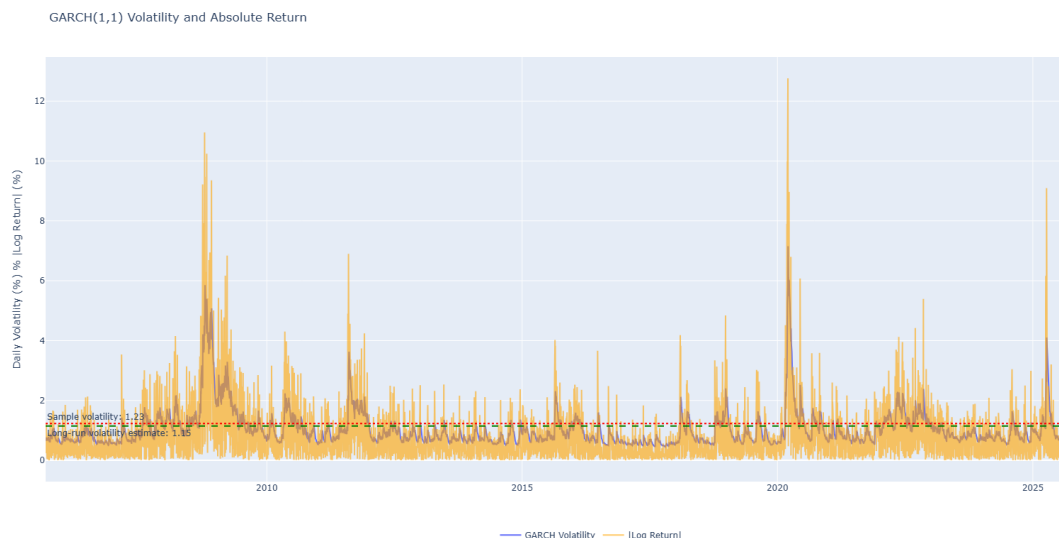


Figure 10: Conditional Volatility and Absolute Log Returns (%)

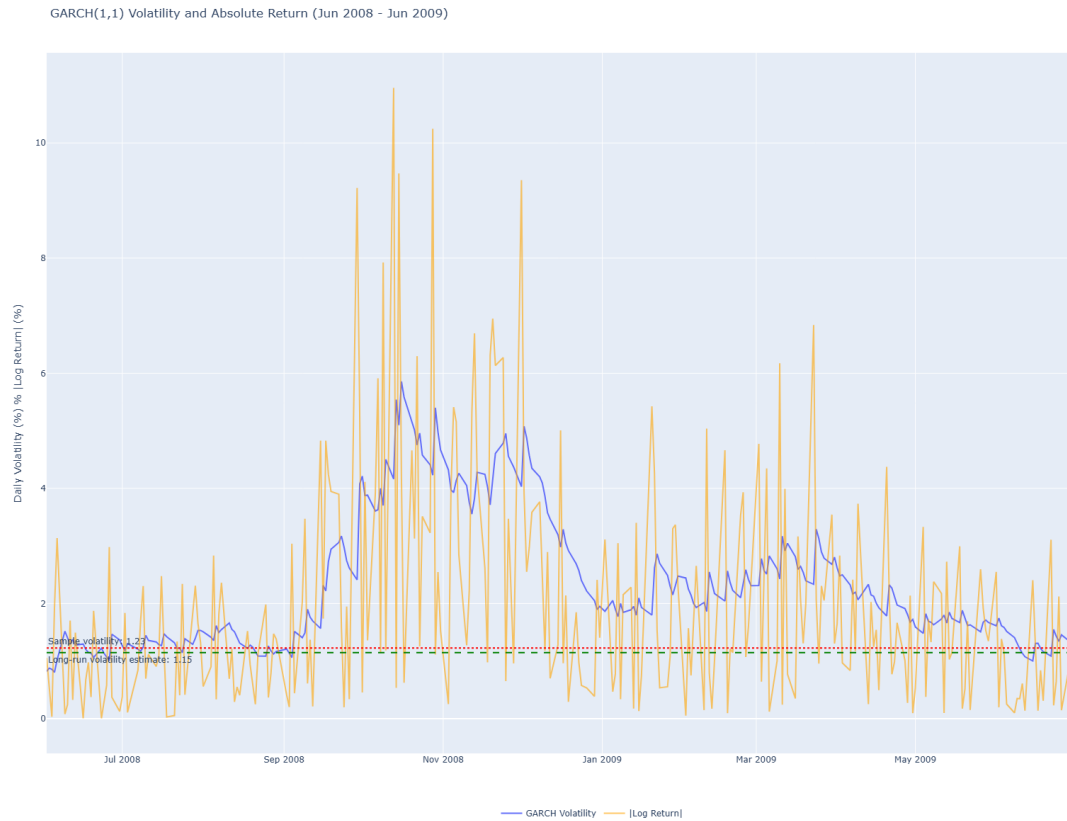


Figure 11: Absolute returns and Volatility: 2008 Crisis

However, the interesting thing happens when we zoom in (preferably somewhere with a big spike in returns). I used the COVID-19 volatility spike on Figure 12:

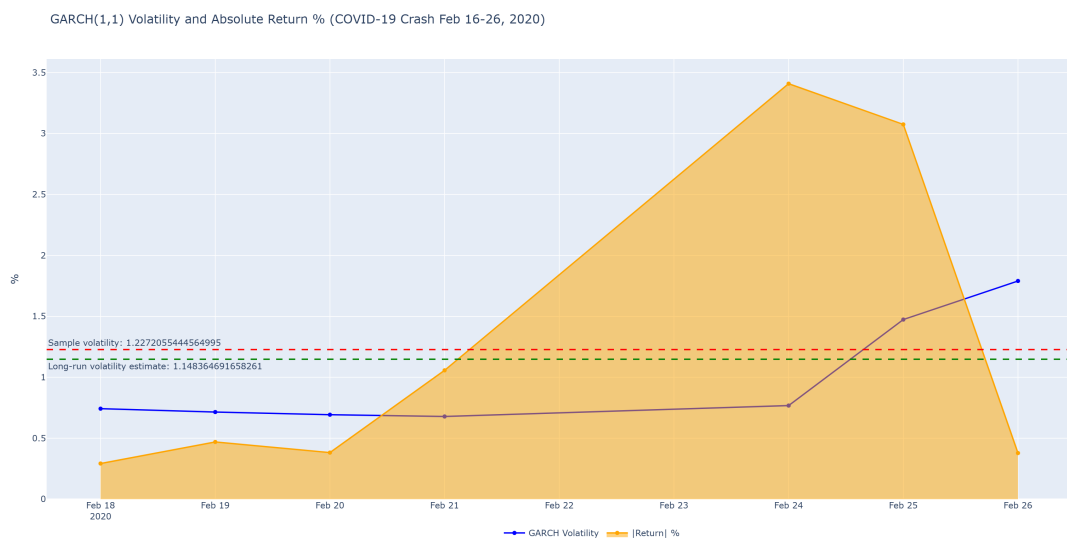


Figure 12: Conditional Volatility and Absolute Returns

We can see that the GARCH volatility is slightly delayed. And this makes sense, since as we explained earlier, the GARCH model is *autoregressive*, i.e. it uses its *own past data*, lagged by 1 period (days in this case).

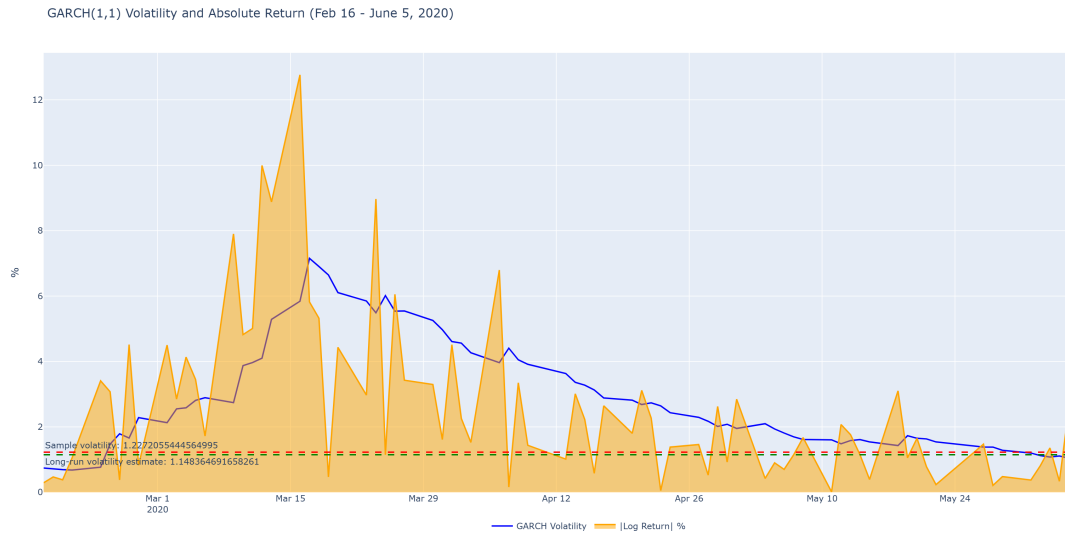


Figure 13: Persistence of volatility

With this other zoomed plot (Figure 13) we can see the effects of having a high $\alpha + \beta$: We see that the GARCH volatility is very persistent (it decays much slower than the returns).

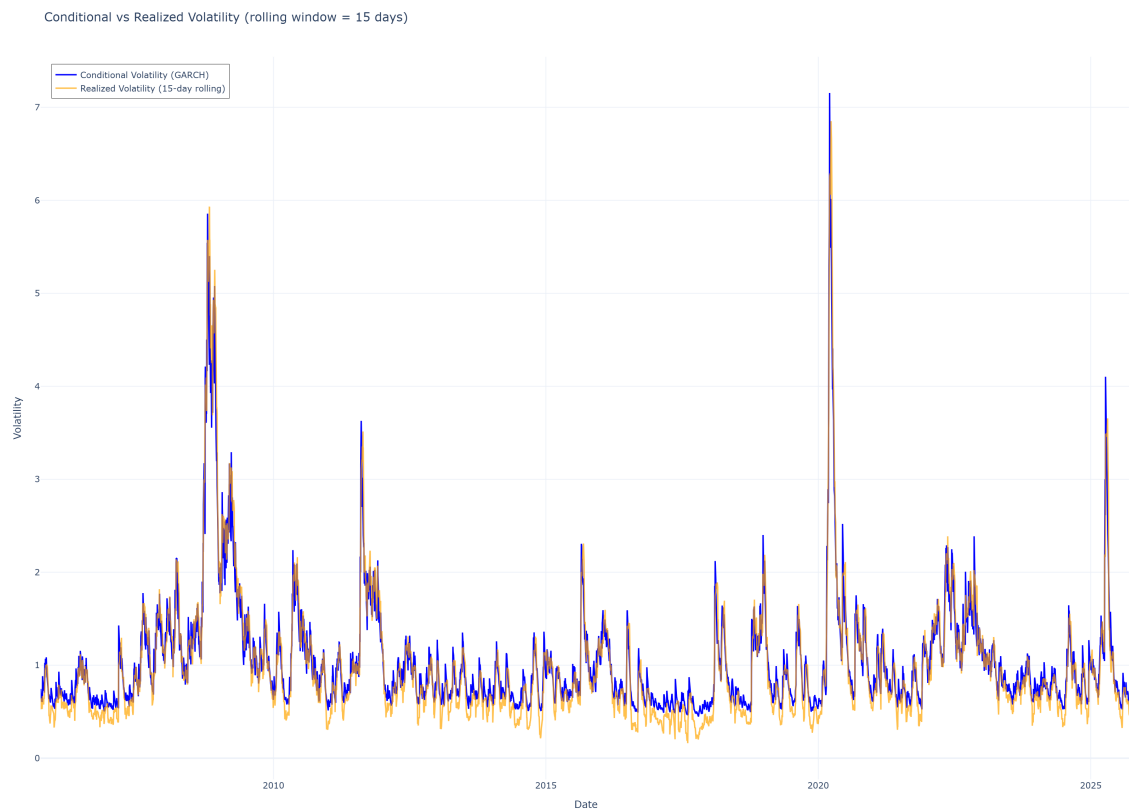


Figure 14: Conditional vs (Smoothed) Realized Volatility

We saw on Figure 10 and Figure 11 that absolute returns are very noisy to compare with conditional volatility. So I plotted the Conditional Volatility against a smoothed (15-day rolling window) Std. Dev.

of the returns, to better appreciate the accuracy of our GARCH model. We see that the conditional volatility follows pretty well the other model

4 Future volatility forecasting with GARCH

Up to this point, we have seen how to estimate *past* volatility values once the model parameters are known. In this section, I will focus on estimating *future* volatility, which is much more relevant for real-life applications, such as hedging risks or evaluating the exposure of a portfolio (as in VaR modelling).

We will explore three forecasting methods:

1. Multi-step (Blind) Forecasting: Forecasting future volatility without access to future returns.
2. Daily Retrain Model: Continuously updating the model with new information.
3. Train-and-Run (Static Parameters): Training the model once and using it to project future volatilities.

4.1 Blind volatility prediction

Sometimes we want to estimate the volatility in N months' time without estimating the market prices first (like being blind to the market). For this, we will implement volatility prediction using GARCH. We will train the model following this procedure:

1. Split the data into $X\%$ training and $(100 - X)\%$ testing.
2. "Train" the GARCH model on the training data (i.e. estimate the parameters).
3. Make predictions for every day of the test period.
4. Evaluate the prediction performance with the test period's realized volatility.

That is, we will predict volatility as if we had *no* information about the market (just the training data) during the test period, and then we will compare it with future test data to see how we did.

But how does the prediction work? By observing the GARCH formula ($\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2$) one would wonder: How does the model get the ε_{t-1}^2 at each iteration without access to the daily return?

Let's see it step by step to identify the issue (and solution!):

On the first iteration, we have the very last ε_0^2 and σ_0^2 of the fitted model. Therefore we have no issues getting σ_1^2 :

$$\sigma_1^2 = \omega + \alpha \varepsilon_0^2 + \beta \sigma_0^2$$

On the second iteration, we want to obtain:

$$\sigma_2^2 = \omega + \alpha \varepsilon_1^2 + \beta \sigma_1^2$$

but we only have σ_1^2 , not ε_1^2 ! Instead of panicking, we use the *estimate* of ε_1^2 i.e., we take the conditional expectation given today's information $\mathcal{I}_0$:

$$E[\sigma_2^2 \mid \mathcal{I}_0] = \omega + \alpha E[\varepsilon_1^2 \mid \mathcal{I}_0] + \beta E[\sigma_1^2 \mid \mathcal{I}_0].$$

(And note: σ_1^2 is already determined by the first step from $\mathcal{I}_0$, so $E[\sigma_1^2 \mid \mathcal{I}_0] = \sigma_1^2$.)

And how do we get $E[\varepsilon_1^2 \mid \mathcal{I}_0]$? As explained before, ε_t is defined as:

$$\varepsilon_t = \sigma_t z_t$$

so $\varepsilon_t^2 = \sigma_t^2 z_t^2 \implies E[\varepsilon_t^2 \mid \mathcal{I}_{t-1}] = E[\sigma_t^2 z_t^2 \mid \mathcal{I}_{t-1}]$.

And z_t is an independent and identically distributed random variable, so it has $\mu = 0$ and $\sigma = 1$. Therefore:

$$\text{Var}(z_t) = E[z_t^2] - (E[z_t])^2 \implies E[z_t^2] = 1.$$

Assuming z_t is independent of $\mathcal{I}_{t-1}$:

$$E[\varepsilon_t^2 \mid \mathcal{I}_{t-1}] = E[\sigma_t^2 \mid \mathcal{I}_{t-1}] \cdot E[z_t^2] = E[\sigma_t^2 \mid \mathcal{I}_{t-1}] \cdot 1 = E[\sigma_t^2 \mid \mathcal{I}_{t-1}].$$

Apply this now to the GARCH recursion for forecasting (with $t = 1$ and conditioning on $\mathcal{I}_0$):

$$E[\sigma_2^2 \mid \mathcal{I}_0] = \omega + \alpha E[\varepsilon_1^2 \mid \mathcal{I}_0] + \beta E[\sigma_1^2 \mid \mathcal{I}_0] = \omega + \alpha E[\sigma_1^2 \mid \mathcal{I}_0] + \beta E[\sigma_1^2 \mid \mathcal{I}_0] = \omega + (\alpha + \beta) E[\sigma_1^2 \mid \mathcal{I}_0]$$

and since, from before, we know that $E[\sigma_1^2 | \mathcal{I}_0] = \sigma_1^2$, we get:

$$E[\sigma_2^2 | \mathcal{I}_0] = \omega + (\alpha + \beta)E[\sigma_1^2 | \mathcal{I}_0] = \omega + (\alpha + \beta)\sigma_1^2$$

This pattern continues for all the next iterations. For $t \geq 1$:

$$E[\sigma_{t+1}^2 | \mathcal{I}_0] = \omega + (\alpha + \beta) E[\sigma_t^2 | \mathcal{I}_0]$$

This is why, when forecasting volatility, the GARCH prediction converges to its long-term variance, since each new step uses the previous forecasted variance as a proxy for the (unseen) squared shock! And this is the mathematical reason why, without new information, the forecasted volatility “forgets” past shocks and drifts toward the unconditional (long-run) volatility level.

Let’s now predict the future volatility of the S&P500. We took 95% of the data to train the model, and 5% to test it. That is, we will predict the volatility of every day for a year.

Training data: 4777 observations (2005-09-26 00:00:00-04:00 to 2024-09-18 00:00:00-04:00)

Test data: 252 observations (2024-09-19 00:00:00-04:00 to 2025-09-22 00:00:00-04:00)

Table 4: Estimated parameters of the GARCH(1,1) model

Parameter	Estimate
μ	0.072132
ω	0.026932
α_1	0.134683
β_1	0.844854

And when compared with the realized volatility, over the test period, we get:

Prediction Performance Metrics:

Mean Squared Error (MSE): 0.6526
Root Mean Squared Error (RMSE): 0.8078
Mean Absolute Error (MAE): 0.5375
Mean Absolute Percentage Error (MAPE): 89.01%
Correlation: 0.1195

Volatility Comparison:

GARCH Long-term Volatility: 1.15 %
Test Period Average Realized Volatility: 0.90 %

Our prediction is terrible! We see that:

- The correlation is very low (0.12), which means that our predictions are not tracking the day-to-day volatility at all.
- The mean absolute error (MAE) is 0.537, which means that on average, our predictions are off by 0.5%. Since volatility is typically between 0.5% and 1.5% most of the time, this is an *ok-ish* error.
- Finally, the mean absolute percentage error (MAPE) is almost 90%, which means that our predictions are off by 90% relative to the realized volatility. For example, if the realized volatility was 0.5% and we predicted 0.25%, then the MAPE would be 50%.

However, these results are expected for a model without access to real information for about one year!

The GARCH(1,1) model is overpredicting volatility (long-term GARCH vol of 1.15 vs. test period avg. realized vol 0.9) and failing to track short-term dynamics (low correlation, very high MAPE). This is exactly what we would expect for a simple GARCH(1,1) when forecasting long horizons.

Let's visualize how such a prediction looks like (Figure 15, Figure 16):

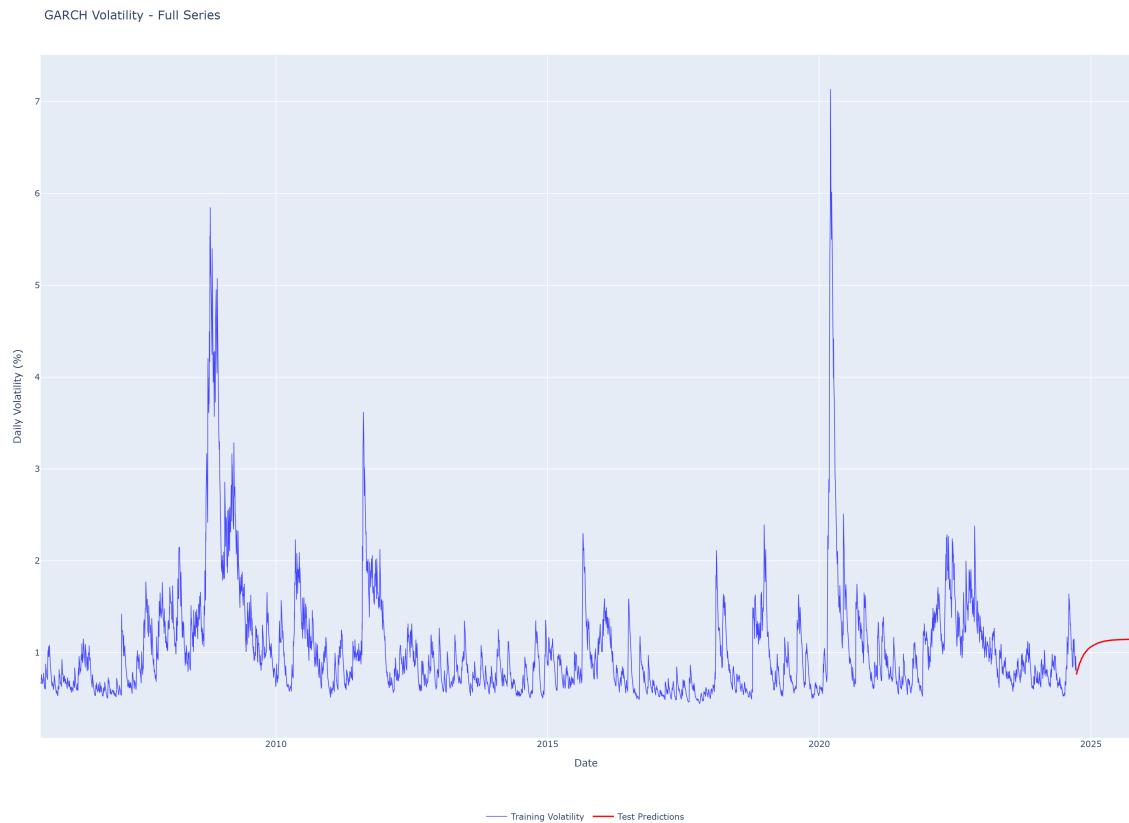


Figure 15: Trained Model Conditional Vol. and 1y Predicted Vol.

On the first plot (Figure 18), we can see the conditional volatility of the fitted model over the train period, and the prediction over the test/forecast period (5%) (without test data). We can see that the prediction is a smooth curve, contrasting with the spiky conditional volatility.

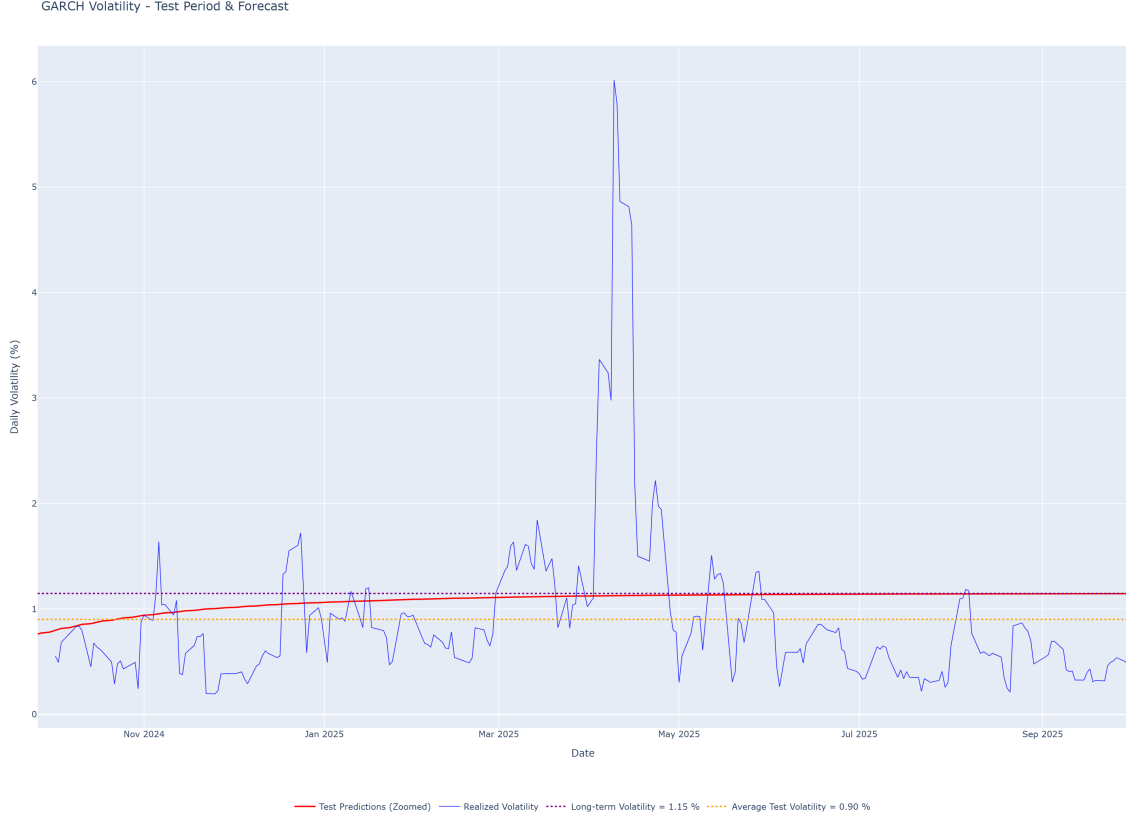


Figure 16: Test period: Prediction vs Realized volatility

But it's on Figure 16 where we can see some interesting things:

1. There is a big difference between the test avg. realized volatility and the model's long-term volatility. This indicates that we trained the model on more volatile data than the one for testing, since the model is constantly overpredicting volatility.
2. Without observations of the market, the prediction converges to the long-term volatility, given by

$$\sigma^2 = \frac{\omega}{1 - \alpha_1 - \beta_1} = \frac{0.027085}{1 - 0.135431 - 0.844168} \approx 1.32$$

$$\sigma = \sqrt{1.32} \approx 1.15 \quad (\text{convert variance into volatility})$$

$$\sigma_{\text{annual}} = \sigma \times \sqrt{252} = 1.15 \times \sqrt{252} \approx 18.29 \quad (\text{annualized long-term volatility})$$

4.2 Continuously Trained GARCH Model

When applying volatility estimation on a daily basis (i.e. a bank trying to predict tomorrow's volatility), we normally have enough time and computational resources to do a 1-day-ahead inference and then retrain the model with the realized return. And repeat the operation for every passing day. This way, we make sure that we are always working with the most up-to-date parameters for the model.

But... is it really important to retrain the model with every passing day? Do the parameters change, or are they rather static.

In this section, I implemented the daily retrain approach:

1. We start with an initial training window (90% of the available data).
2. Make a 1-day ahead inference of σ_{t+1}^2 (using σ_t^2 and ε_t^2).

3. Retrain the model with the newly observed return.
4. Repeat the process for each subsequent day.

This approach resembles way more to the real-world forecasting where models are continuously updated with new information. However, it is very compute-intensive (1 model fitting per period).

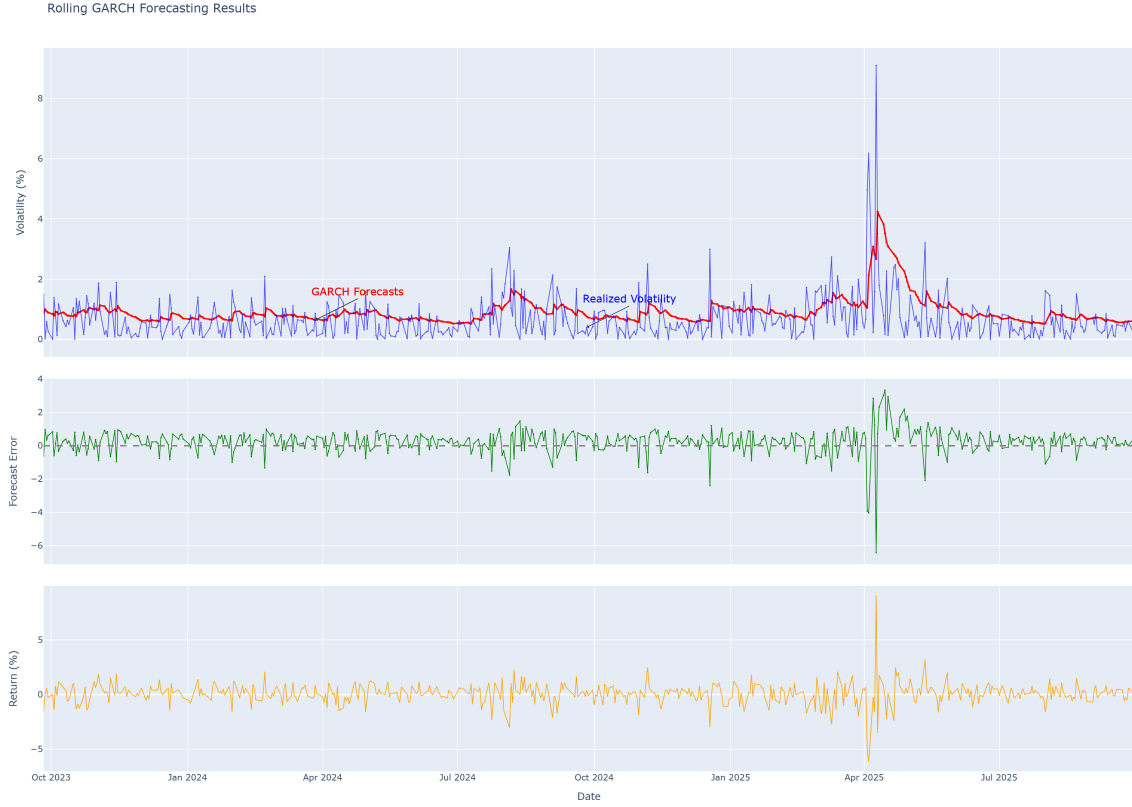


Figure 17: Daily predictions, Error, Log Returns (%)

But... do parameters change much over time when retraining the model every day? Over the 505 trainings we did, these are the results we obtained:

Parameter	Mean	Std	Min	Max	% Change
ω	0.028007	0.000934	0.026533	0.030406	5.894782
α_1	0.139717	0.003957	0.134889	0.148304	7.523803
β_1	0.841077	0.003241	0.833831	0.846025	-1.051784
Persistence	0.980794	0.001118	0.978314	0.983777	0.141535
Long-term volatility	1.208951	0.040741	1.174678	1.313638	6.780480

Table 5: Summary Statistics of GARCH(1,1) Parameters

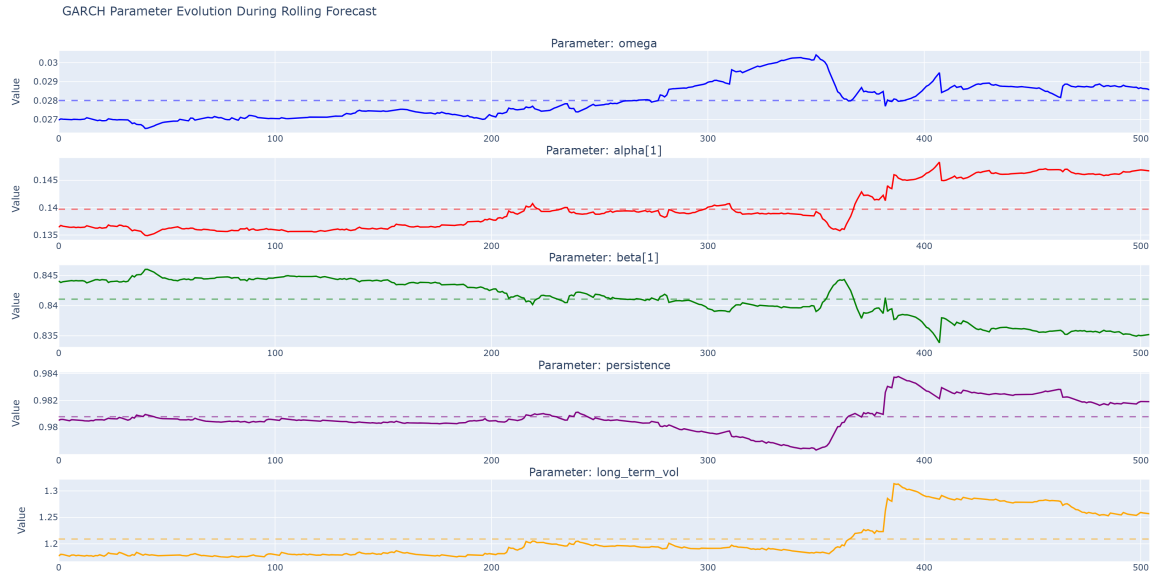


Figure 18: Parameter evolution over time

Figure 18 and Figure 19 show the following:

1. The parameters α and β appear to be (somewhat) inversely correlated.
2. The long-term volatility reacts quickly to shocks, largely driven by the α component.
3. Persistence remains relatively stable throughout the training period.
4. Over time, the model increasingly emphasizes the sequence of shocks (progression-wise) rather than the magnitude of past volatility values.

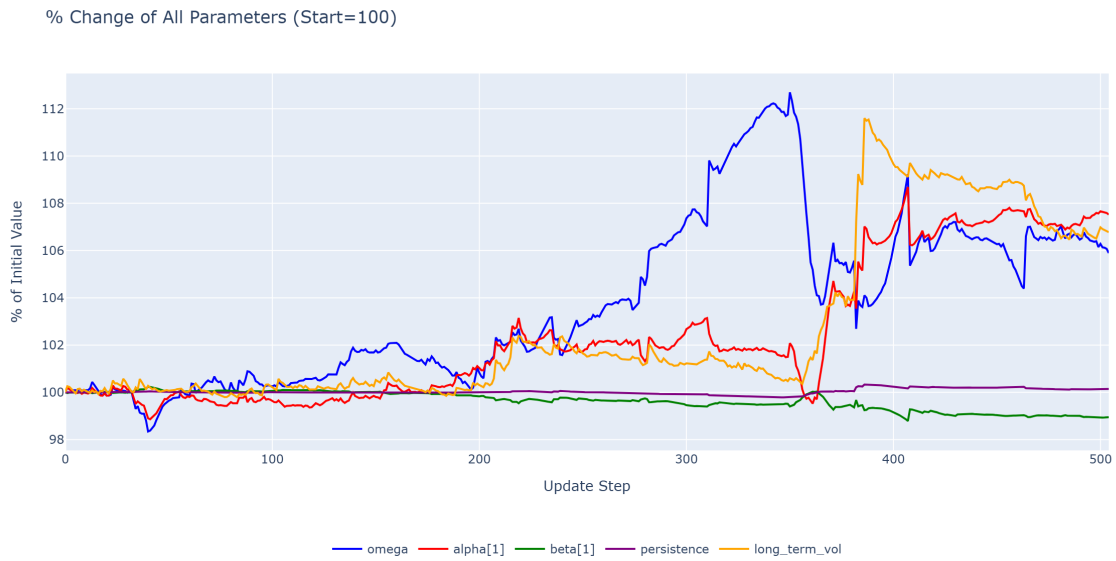


Figure 19: Parameter % Change over time

4.3 Train-and-Run Approach

There are some instances in which we cannot retrain the model for every period (for example, with High Frequency Trading algorithms), because we have no time, computational resources or we simply don't want to overfit the model with new information (we saw earlier how quickly parameters can change given strong volatility periods).

We previously mentioned that after obtaining the parameters of the model, we could "go back" to very same data we had trained the model on, and use the formula and the realized shocks of every passing day to calculate the conditional volatility of that day (and use it to compute the one for the next day).

But what will happen if we get the parameters of the model by fitting the model only with past data, and then try to infer future volatility with the outdated parameters? i.e. what if we could only train the model once?

The procedure would be as follows:

1. Train the GARCH model on the training dataset (a subset of the full dataset).
2. Extract the model parameters after training.
3. Use these parameters to make future predictions on the test dataset without retraining, applying the standard recursion formula.

First, we fit the model on 30% of the available data, obtaining the parameters on Table 6:

Table 6: GARCH Parameters (Train)

Parameter	Value
μ	0.052940
ω	0.017542
α_1	0.094959
β_1	0.897258

We see that it has a very high persistence ($\alpha + (\beta)$), meaning that the modeled volatility will be very "sticky".

Then we recursively apply the formula explained previously with the daily realized returns of each day:

$$\sigma_t^2 = \omega + \alpha_1 \epsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2, \quad (1)$$

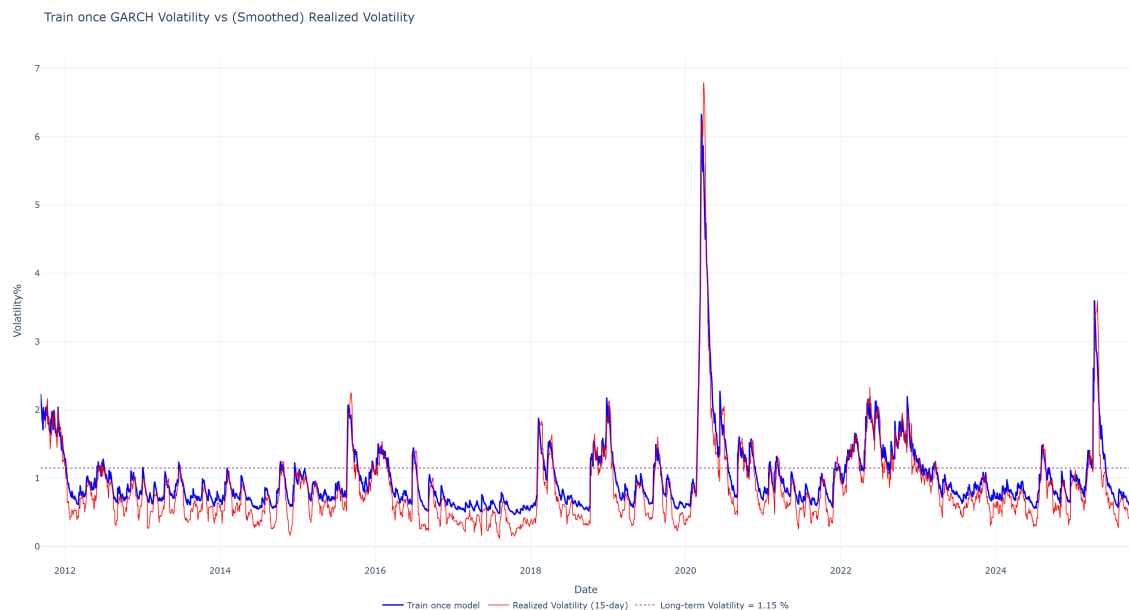


Figure 20: Trained Once vs (Smoothed) Daily returns

When comparing against the 15-day-smoothed daily volatility (i.e. the rolling window standard deviation of the previous 15 daily returns - the (μ) , we get the following Goodness-of-Fit metrics:

MSE: 0.0431 RMSE: 0.2077 MAE: 0.1728 MAPE: 31.4854 Correlation: 0.9734

So, is it really worth it to retrain the model daily like we did earlier, or is it fine if we keep using the same parameters for a long period? To check it, I compared the following:

1. A model trained on the first 30% of the data and never again refitted.
2. A model initially trained on the first 30% of the data, but subsequently retrained daily on the remaining 70%, so that the last fitted model incorporated all available data.

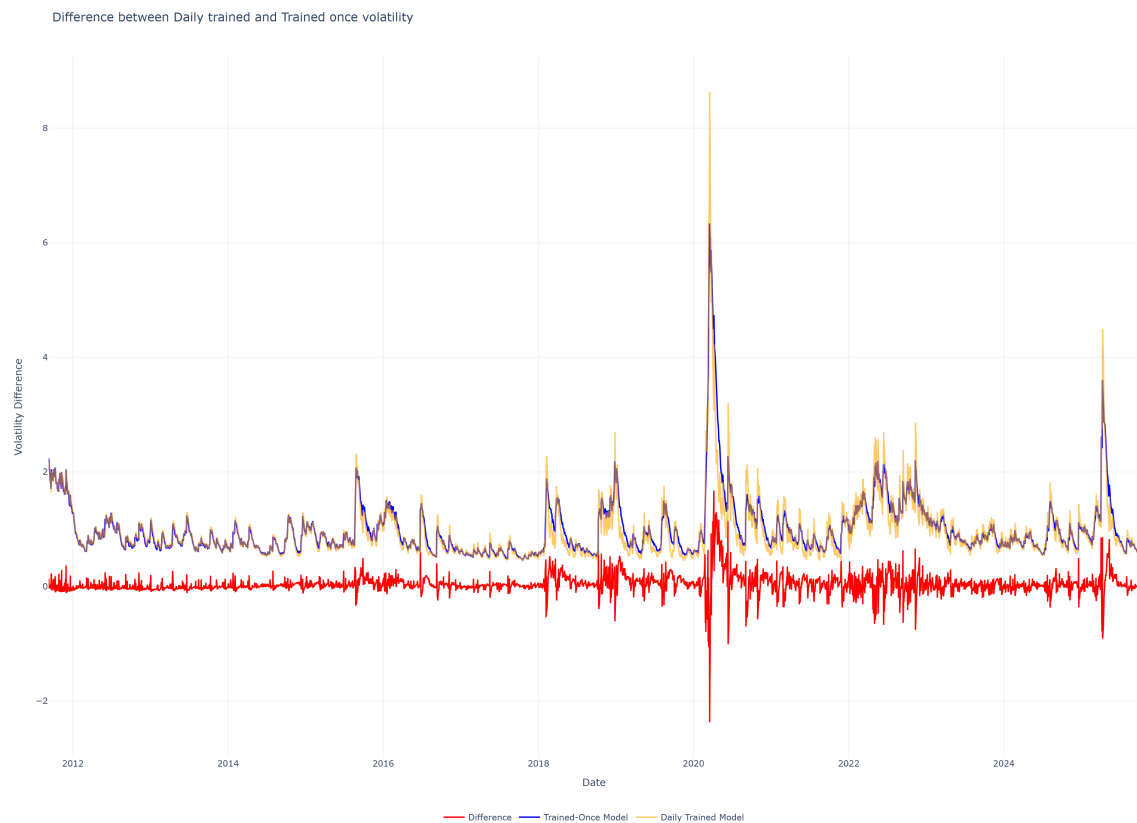


Figure 21: Daily trained vs Trained Once Volatilities

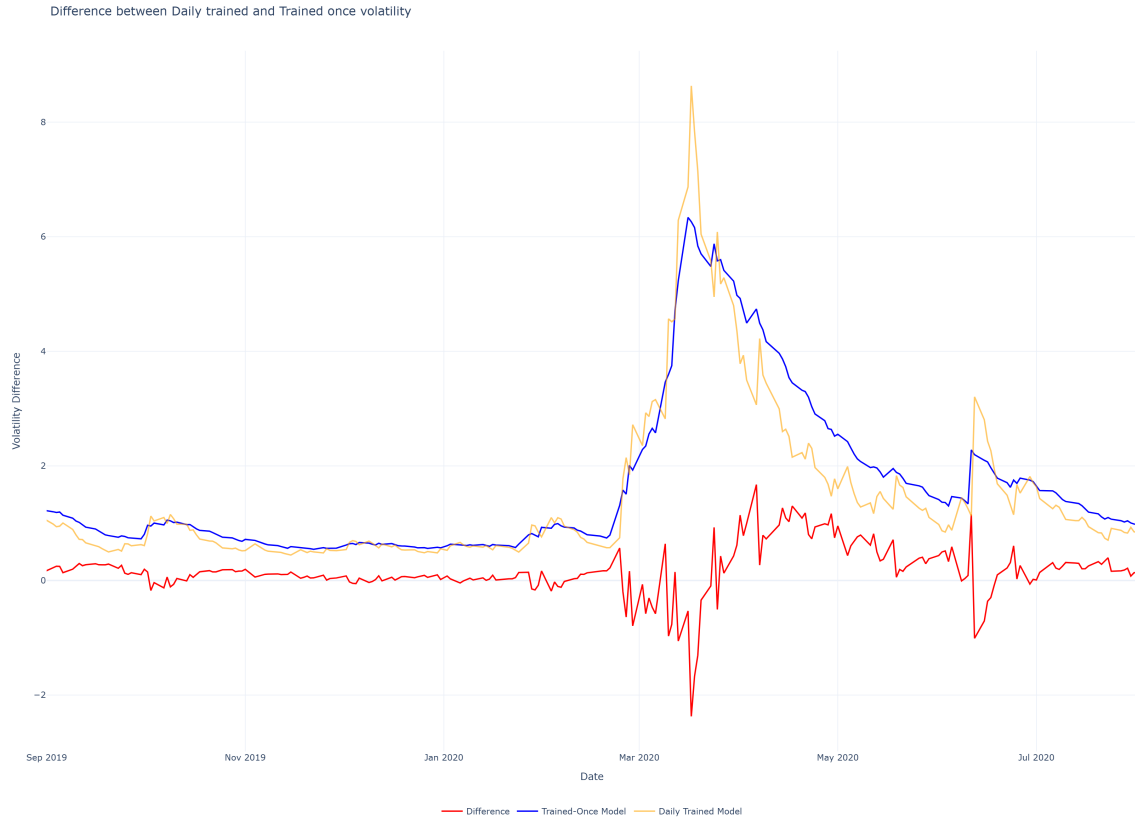


Figure 22: Zoomed Difference between Daily Trained and Trained Once model volatility

Figure 21 shows us the comparison between the Conditional Volatilities of each method (they are conditional because they incorporate the data of up to $t-1$). We observe that they are mostly identical, except for their behaviour on volatility spikes (Figure 22), where they differ significantly.

When comparing both models against the 15-day realized volatility (Std. Dev. of the log returns of the previous 15-days), we get the following metrics:

Model Performance Comparison		
Metric	Train Once	Daily Retrain
MSE	0.147926	0.108244
RMSE	0.384612	0.329004
MAE	0.311638	0.274661
MAPE (%)	73.69	65.41
Correlation	0.890176	0.924597

Performance Improvement (Daily Retrain vs Train Once):

MSE Improvement: 26.83%
 MAE Improvement: 11.87%
 MAPE Improvement: 11.23%
 Correlation Improvement: 3.87%

So we arrive at the conclusion that retraining the model daily does help at getting a better representation of the volatility.

4.4 Comparison of Volatility Estimation Methods

So far, we have seen three different methods for estimating the volatility at a given point in time:

1. Training the model on the full dataset and then using the estimated parameters to compute the conditional volatility.
2. Initially training the model on a smaller dataset, then retraining it each day to infer the next day's volatility.
3. Training the model once and never refitting it again, using the recursive formula to obtain the next day's variance.

But how do these three approaches compare? Figure 23 shows the *Conditional Volatility* (obtained by training the model with the entire dataset), alongside the *Daily Retrained Model* and the *Trained-Once Model* for comparison.

Conditional Volatility Comparison of GARCH Models



Figure 23: Comparison of the Three Volatility Estimation Methods (Zoomed View)

It is important to note that in all three cases, the volatility is *conditional*, since the next-day variance is obtained using the previous day's variance and shocks. Therefore, the volatility at time t is conditioned on the information available at $t - 1$ in all three models.

4.5 Comparing Conditional Volatility with VIX

As we saw earlier, the VIX is an index that measures the expected annualized volatility of the US market over the next 30 days. It is a measure of the expectation of the total volatility that might happen on the period starting in 30 days time, and ending in 1 year and 30 days time. The higher VIX, the higher the volatility is expected to be. It is calculated from option prices of the SP500 (i.e. if investors pay more for the options, more volatility is expected).

Since our GARCH also measures volatility, let's compare them:

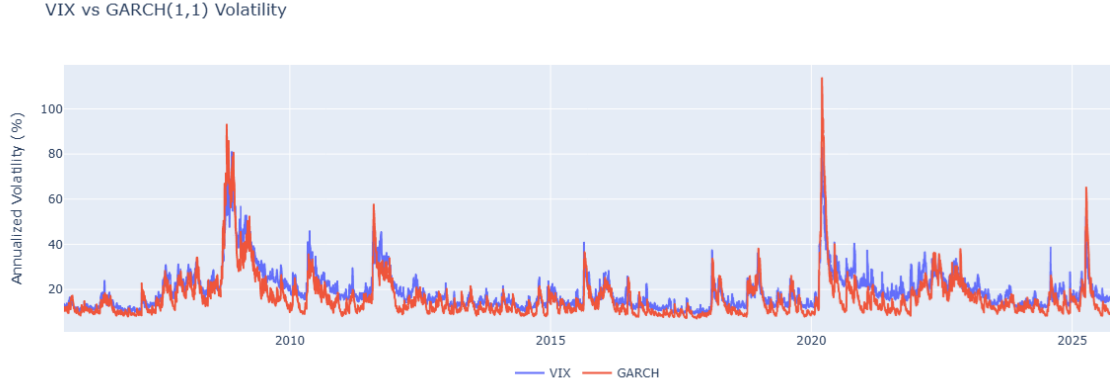


Figure 24: Vix vs GARCH volatility

Figure 24 shows that our GARCH(1,1) model is effectively capturing the market's expectation of future volatility as represented by the VIX index.

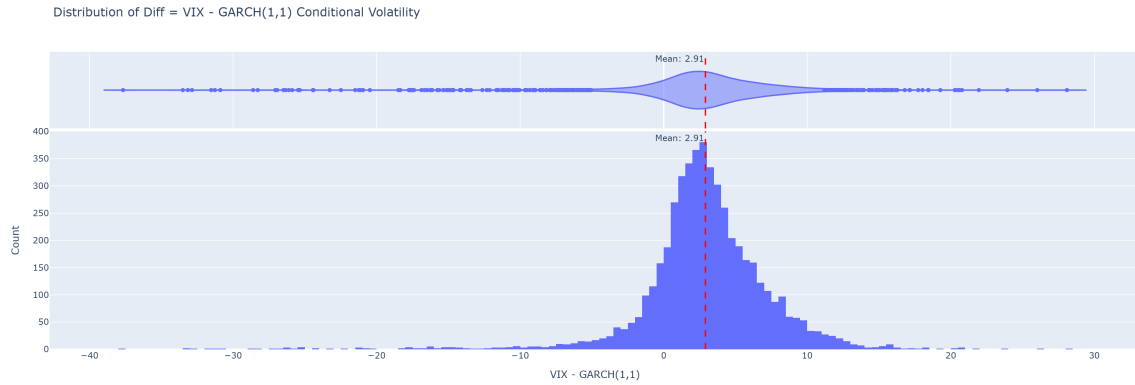


Figure 25: VIX - GARCH distribution

Table 7: Summary Statistics

Statistic	Value
Mean	2.91
Std. Dev.	4.64
Min	-37.66
25%	1.04
50% (Median)	2.86
75%	5.10
Max	28.09

Looking at the distribution (Figure 25) of the errors and the graph above, we can interpret as follows: The long left tail indicates that we sometimes overestimate the VIX significantly, while the shorter right tail suggests that our underestimations are generally smaller in magnitude, but much more frequent! If we look at Figure 24, we can see that the conditional volatility is almost always below the VIX (mostly in periods of calm), but we go over the index on large spikes (which are less frequent but make our model overreact, plus the high persistence of our model makes it stay high for a while).

5 Regime Classifiers

The GARCH model returns many volatility values. However, the general public doesn't understand the difference between a σ of 0.04 and 0.05.

The solution to this is to classify these values in more abstract levels (like high, medium and low). These levels of volatility are called Regimes, and identifying which regime we were into on past (or even current) periods is called Regime Classifying. On this practice we will use two methods: Hidden Markov Models (HMM) and Markov Switching Autoregressive Model (MSAR), both linked to Markov Chains.

5.1 Introduction to Markov Chains

A Markov Chain is a system that transitions from one state to another, where the probability of moving to the next state depends only on the current state, not on the sequence of events that preceded it (i.e. we only care about today to tell what we'll do tomorrow). That is, it has no memory of the past (only today matters).

So for every state, we have a probability of going to any other state (including itself). These probabilities can be stored in a matrix called the transition matrix.

$$P = [P_{ij}], \quad \text{where } P_{ij} = P(X_{t+1} = s_j \mid X_t = s_i)$$

And the memory-less property can be expressed as:

$$P(X_{t+1} = s_j \mid X_t = s_i, X_{t-1}, \dots, X_0) = P(X_{t+1} = s_j \mid X_t = s_i)$$

That is, the probability of transitioning to another state depends only on the current state (X_t) and not on any previous states ($X_{t-1}, \dots, X_0$). So we don't mind if we lack some farther past value.

The classical example is as follows:

Suppose we have two states:

- s_1 = Sunny
- s_2 = Rainy

and the transition matrix is:

$$P = \begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix}$$

- The rows correspond to the current state (today).
- The columns correspond to the next state (tomorrow).
- Each row sums to 1, because they represent probabilities of all possible next states.

So once we have the matrix, we can say that:

- if today is Sunny (s_1):
 - probability of tomorrow being Sunny = 0.8
 - probability of tomorrow being Rainy = 0.2
- if today is Rainy (s_2):
 - probability of tomorrow being Sunny = 0.4
 - probability of tomorrow being Rainy = 0.6

5.2 Hidden Markov Model

We can start seeing what is the connection to Hidden Markov models (which is a model that assumes the memoryless property of Markov chains). On the HMM, the states are always hidden (i.e. we don't know what state we are in). Continuing with the classic example, we don't know if today is sunny or rainy, but we have some observable data that depends on the state (e.g., if people are carrying umbrellas, it is more likely to be rainy).

But the “we only care about the present” philosophy is maintained. That is, the hidden states behave as a Markov Chain.

For a HMM, we have the following components:

1. States set: $S = \{s_1, s_2, \dots, s_N\}$ (hidden states)
2. Observations set: $O = \{o_1, o_2, \dots, o_M\}$ (observable outputs)
3. Transition probabilities: $A = [a_{ij}]$ where $a_{ij} = P(X_{t+1} = s_j \mid X_t = s_i)$
4. Emission probabilities: $B = [b_{jk}]$ where $b_{jk} = P(Y_t = o_k \mid X_t = s_j)$
5. Initial state distribution: $\pi = [\pi_i]$ where $\pi_i = P(X_1 = s_i)$

Let's ground it with the rainy/sunny example: - States: $S = \{s_1 = \text{Sunny}, s_2 = \text{Rainy}\}$ - Observations: $O = \{o_1 = \text{No Umbrella}, o_2 = \text{Umbrella}\}$ - Transition matrix:

$$A = \begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix}$$

- Emission matrix:

$$B = \begin{bmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{bmatrix}$$

The emission matrix tells us the probability of observing an umbrella or not, given the weather state. For example, if it's sunny, there's a 90% chance of seeing no umbrella, and if it's rainy, there's a 80% chance of seeing an umbrella, and a 20% chance of seeing no umbrella.

In our use case, the umbrellas (observations) are the volatility values we get by doing the filtering / or 1day forecast with GARCH, and the hidden states are the volatility regimes (e.g., high, medium and low volatility).

But how does the HMM work in our case? Basically the model is initialized with random values for the means, variances, transition probabilities, etc... Then, it uses the [Baum-Welch algorithm](#) (that combines two other algorithms until convergence or hitting the limit: forward and backward) to iteratively adjust these parameters to maximize the likelihood of the observed data (i.e. the volatilities we got from GARCH).

After fitting, the model obtains N states (3 in our case) clusters with means and variances, and also the transition matrix. Here are the results for our experiment:

```
State means (mus): [10.56188403 17.07205403 34.91428278]
State standard deviations (sigmas): [ 1.43257215  2.91549997 15.72113009]
Transition matrix (transmat):
[9.74158095e-01 2.45522167e-02 1.28968871e-03]
[3.10906763e-02 9.58135226e-01 1.07740981e-02]
[1.13255006e-54 3.45442454e-02 9.65455755e-01]
```

This data tells us that:

- **State means:**

- State 1 (Low Volatility): $\mu = 10.56$ (low volatility regime)
- State 2 (Medium Volatility): $\mu = 17.07$ (medium volatility regime)
- State 3 (High Volatility): $\mu = 34.91$ (high volatility regime)

- **State standard deviations:**

- State 1 (Low Volatility): $\sigma = 1.43$ (very stable)
- State 2 (Medium Volatility): $\sigma = 2.92$ (moderate variability)
- State 3 (High Volatility): $\sigma = 15.72$ (very unstable)

• **Transition matrix (P):**

$$P = \begin{bmatrix} 0.9742 & 0.0246 & 0.0013 \\ 0.0311 & 0.9581 & 0.0108 \\ 0.0000 & 0.0345 & 0.9655 \end{bmatrix}$$

- High persistence in all regimes (the diagonal entries are very close to 1).
- Directly going from Low to High (and viceversa) is almost impossible.

So now let's plot it:



Figure 26: HMM Volatility Regimes and GARCH

Each column on the plot above (Figure 26) has a colour, or several. When a column has a single color, it means that the probability of that state being the regime is 100%. If it has two colors, it means that the model is unsure, therefore it might assign 70% to one color (one regime) and 30% to another one. The same happens when there are three colours on a single column.

Note that it doesn't matter which color the GARCH line touches.

This plot is the equivalent of writing that for each column (i.e. each date), there are probabilities [Pr(low vol), Pr(mid vol), Pr(high vol)]

Each column represents a row of the observation matrix, for each value of GARCH at a point in time. It is hard to see, but each column can contain the three probabilities of the states. However, the fact that it is hard to see tells us that the model switches with a lot of confidence between states. This is better appreciated with figure 27.

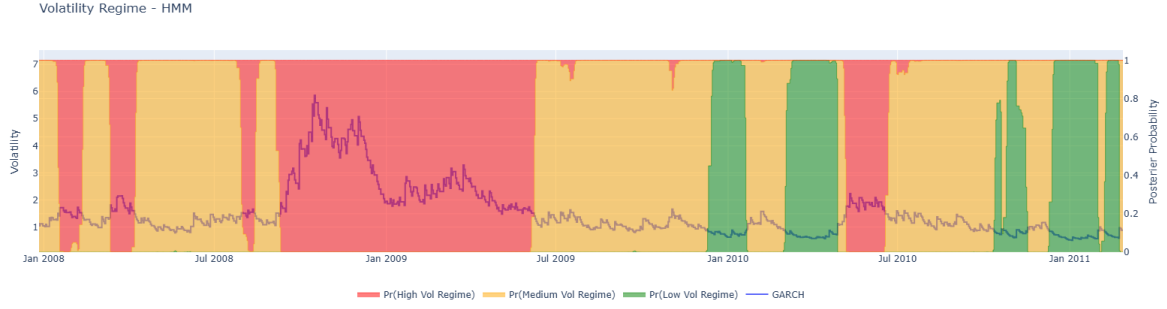


Figure 27: Zoomed HMM Volatility Regimes and GARCH

We can also see that the model makes clusters of regimes (we can observe this with the 2008 financial crisis, the 2011 European debt crisis, the 2020 COVID-19 crisis, etc.). That is, the model doesn't just indicate that there is high volatility on one day where the conditional volatility spikes, and then it returns to normality. It waits until the volatility settles to change the regime.

5.3 Markov Switching Autoregression Model

However, we also see with the previous plot that the model is very sure when it changes the regime. In real life, this is not the case: We don't wake up one morning and say "High volatility is over! Now we are in low volatility!" Instead, we transition slowly between regimes.

To account for this property, we use the Markov Switching Autoregression Model ([MSAR](#)). This model is very similar to the HMM, but it forces the transition matrix to have high values on the diagonal (i.e. high probability of staying in the same state). This way, we force the model to be more "sticky" with the regimes. Note that the *memoryless* Markov property is still applied (hence the name), but we are also making past values influence the model (that's why it's called *autoregressive*).

In the MSAR model, the observed series y_t (in this case, the log returns - μ , i.e. the shocks ε) follows an autoregressive process whose parameters depend on the underlying regime s_t at each moment in time:

$$y_t = \phi_{s_t} y_{t-1} + \varepsilon_t, \quad \varepsilon_t \sim N(0, \sigma_{s_t}^2)$$

Here:

- $s_t \in \{1, 2, \dots, K\}$ is the hidden regime at time t .
- ϕ_{s_t} is the autoregressive coefficient in regime s_t .
- $\sigma_{s_t}^2$ is the variance of the residuals in regime s_t .

Because of the configuration of the MSAR that I used, there is no constant term, so each regime's conditional mean depends only on the autoregressive coefficient and the mean of the series (which, as we saw, is approximately 0).

The estimated transition matrix is:

Transition matrix:

```
[9.70925205e-01 3.33731743e-02 7.50907567e-05]
[2.87771831e-02 9.60406339e-01 3.63966343e-02]
[2.97612081e-04 6.22048624e-03 9.63528275e-01]
```

and the state standard deviations (volatilities) are:

State standard deviations (sigmas): [0.0053 0.0118 0.0307]

and the autoregressive coefficients are:

[-0.0519 -0.0440 -0.1970]

This means that, in each regime, returns follow an [AR\(1\)](#) with different persistence and volatility levels. Formally, for three regimes, we can write:

$$y_t = \begin{cases} \phi_1 y_{t-1} + \varepsilon_t, & \text{if } s_t = 1, \quad \varepsilon_t \sim N(0, \sigma_1^2) \\ \phi_2 y_{t-1} + \varepsilon_t, & \text{if } s_t = 2, \quad \varepsilon_t \sim N(0, \sigma_2^2) \\ \phi_3 y_{t-1} + \varepsilon_t, & \text{if } s_t = 3, \quad \varepsilon_t \sim N(0, \sigma_3^2) \end{cases}$$

The negative AR(1) coefficients we obtained after fitting the model mean that returns tend to move back toward their average after a shock(mean reversion). This effect becomes stronger during high-volatility periods (where $\phi_3 = -0.197$). Therefore, we see that high-volatility periods are less persistent in returns and more mean-reverting, while low-volatility periods tend to show more stable behaviours.

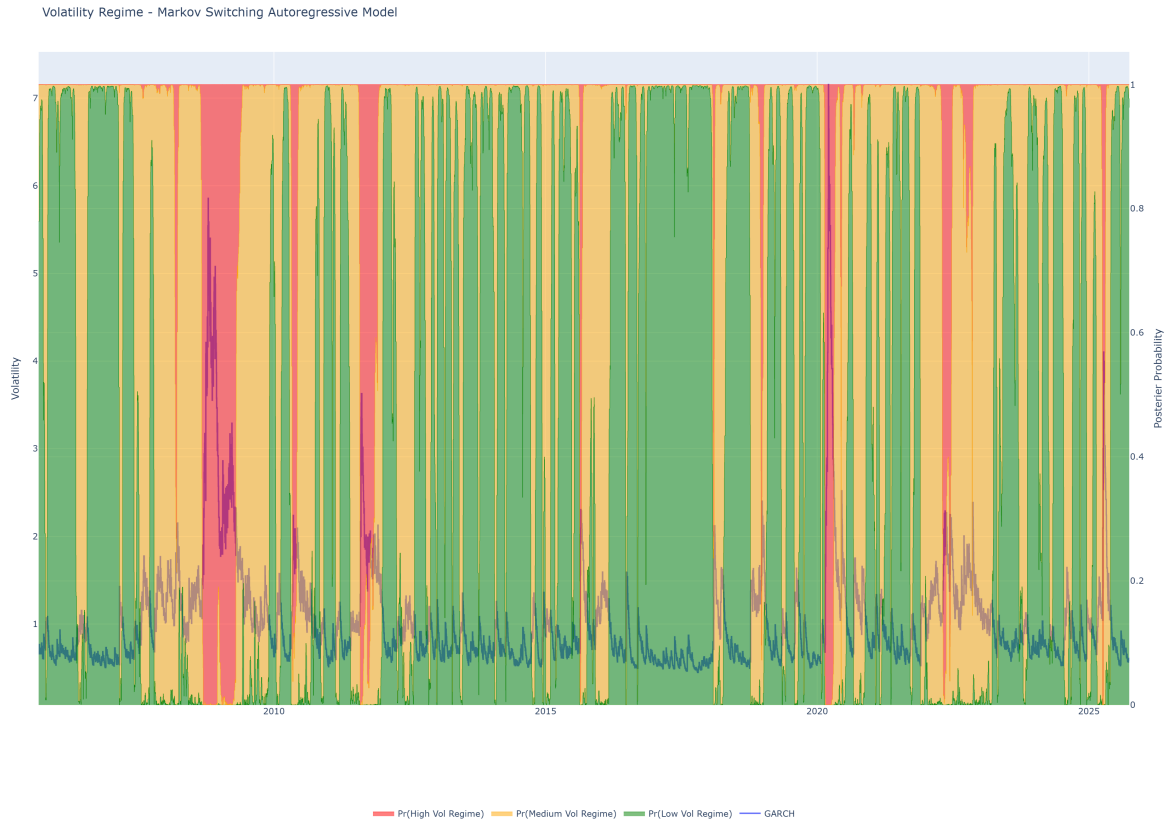


Figure 28: MSAR Regime Classifier GARCH

Volatility Regime - Markov Switching Autoregressive Model

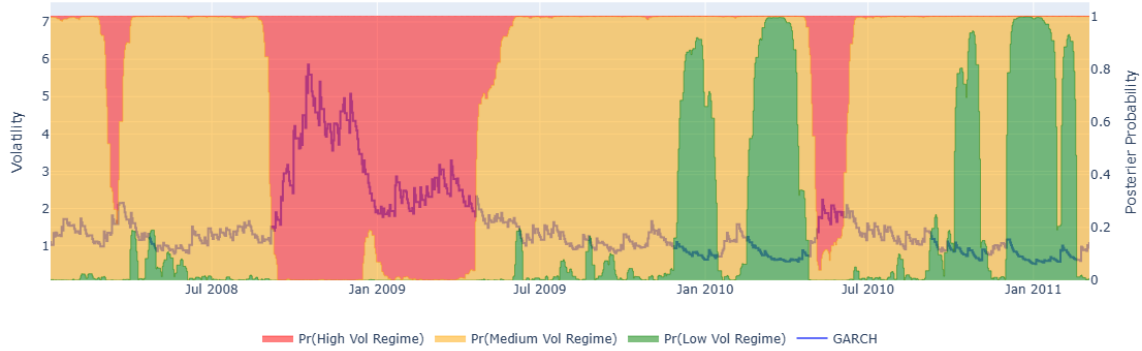


Figure 29: Zoomed MSAR Regime Classifier & GARCH

And exactly as expected, Figure 29 shows that the model sticks more to the states (i.e. there are more “curves”, which mean that the probabilities of being in a state change more slowly).

5.4 Comparison

A useful method for comparing how well each model fits the data is the log-likelihood. The log-likelihood measures how probable the observed data is, given the model parameters. A higher log-likelihood indicates a better fit to the data.

log-likelihood of HMM: -12596.045936794613

Transition Matrix of HMM:

```
[[9.74158095e-01 2.45522167e-02 1.28968871e-03]
 [3.10906763e-02 9.58135226e-01 1.07740981e-02]
 [1.13255006e-54 3.45442454e-02 9.65455755e-01]]
```

Log-likelihood of MSAR: 16446.800669230575

Transition Matrix of MSAR:

```
[[9.70666336e-01] [3.33071545e-02] [2.11475002e-06]
 [2.88565606e-02] [9.60615885e-01] [3.83404736e-02]
 [4.77103580e-04] [6.07696087e-03] [9.61657412e-01]]
```

We see that both models say that states are very persistent (over 95% of staying). The higher log-likelihood value of the MSAR indicates that it is a better fit for the input data (i.e. the GARCH volatilities). This is expected, since the MSAR model accounts for the persistence of volatility regimes, which is a known characteristic of financial markets.

However, we can see that the MSAR model is not necessarily more “sticky” than the HMM. In fact, the HMM has a higher probability of staying in the low volatility regime and the high volatility regime. The MSAR model has a higher probability of staying in the medium volatility regime. This means that the MSAR model is more likely to switch between low and high volatility regimes, while the HMM model is more likely to stay in the same regime.

6 Conclusion

This practice has been much more than just an exercise in applying volatility models. It has been a way for me to truly understand how modern methods like GARCH and regime-switching models work in practice, and what they can (and cannot) tell us about financial markets.

Paradoxically, I intended to make it not very rigorous (I wanted to focus on understanding the topic, rather than proving everything around it), but I ended up discovering many statistical tests that help check the assumptions behind these models and to choose between them.

The real value, though, has gone beyond the models themselves. Working through the material has made me more aware of the assumptions hidden in the simplest things we often take for granted, like how we define returns, or what we assume about prices and distributions. It has also shown me how research in finance differs from pure mathematics: while math often focuses on very intricate theorems, finance requires constant confrontation with messy, imperfect data.

Along the way, I've revisited key probability theory, explored hypothesis testing, and discovered new areas of quantitative finance. I've also seen how GARCH models can be useful in real applications, while at the same time realizing their limitations. Most importantly, I've learned the value of persistence: to keep digging until I find an explanation that makes sense to me, even when the literature feels too dense or overly technical.

In the end, this notebook has been both a learning tool and a personal exploration by helping me connect theory with practice, and motivating me to keep asking questions that lead to deeper understanding.

7 References

- [Maximum Likelihood in the GJR-GARCH\(1,1\) Model \(CrossValidated\)](#)
- [Maximum Likelihood Estimation \(Bookdown\)](#)
- [Markov Chains \(Brilliant.org\)](#)
- [Markov Chains \(YouTube video\)](#)
- [ARCH Definition \(Investopedia\)](#)
- [Market Regime Detection Using HMMs \(QuantStart\)](#)
- [SSRN Paper on Volatility Modelling](#)
- [ARCH Test Explained \(NumXL\)](#)
- [Understanding VIX with GARCH and ARIMA \(Medium\)](#)
- [ARCH Model \(FSB Miami\)](#)
- [arch.univariate.HARX Documentation](#)
- [GARCH Forecasting under Different Distribution Assumptions \(Willhelmesson, 2006\)](#)
- [Gregory Gundersen – Log Returns](#)
- [Returns vs Log Returns \(Quant.SE\)](#)
- [Predicting Volatility with HAR Models \(SR-SV Blog\)](#)
- [Practical Time Series Analysis – Code Repo \(GitHub\)](#)
- [HMMLearn Documentation](#)
- [ECB Volatility Regime \(2018\)](#)
- [ARCH and Regime Changes \(ScienceDirect\)](#)
- [Kim, Nelson, Startz \(1998\) – Three-State Variance Switching](#)
- [Statsmodels – Markov Autoregression Example](#)
- [Statsmodels – Markov Regression Example](#)